# Introduction to **Django**

The full-featured web application framework for Python

# Introduction

About the course

# Arianne Dee

- Full-stack software developer and freelancer
- Passion for user-focused design for the public good
- Has taught thousands of students of all ages:
  - Engineers Without Borders
  - Canada Learning Code
  - O’Reilly and Pearson
- Many videos and live trainings on O’Reilly
- Get developers of all experiences get up-to-speed with Python

# About this course

- Adapted from a 4-hr live training
- Code-along – Introduction to Django
  - Follow at your own pace
- Example project – Trivia Game
  - Intermediate topics discussed in slides
  - References to project code

# Introduction to Django

1. Introduction
2. Get Started With Django
3. Write a Django App
4. Basic Concepts
5. Intermediate Concepts
6. Summary

# How to use this video

- Code along with **lessons 2, 3 & 4** if you are just starting out with Django and want a short intro

- Follow along with other project-based tutorials to get more hands-on practice

- **Lesson 5** introduces intermediate concepts that you can apply to your own project
  - Example app hosted on PythonAnywhere:
    - http://trivia-ariannedee.pythonanywhere.com/

# Introduction to Django

1. **Introduction**
   - Course requirements
   - Overview of Django
2. Get Started With Django
3. Write a Django App
4. Basic Concepts
5. Intermediate Concepts
6. Summary

# Introduction to Django

1. Introduction
2. **Get Started With Django**
   - Learning resources
   - Before you start
   - Create a project
   - Default project files
   - Create a view
3. Write a Django App
4. Basic Concepts
5. Intermediate Concepts
6. Summary

# Introduction to Django

1. Introduction
2. Get Started With Django
3. **Write a Django App**
   - Overview of Django apps
   - Write a Visits app
4. Basic Concepts
5. Intermediate Concepts
6. Summary

# Introduction to Django

1. Introduction
2. Get Started With Django
3. Write a Django App
4. **Basic Concepts**
   - Settings
   - Django ORM
   - Views
   - URLs
   - Templates
   - Admin
5. Intermediate Concepts
6. Summary

# Introduction to Django

5. **Intermediate Concepts**
   - Django architecture
   - Django project structures
   - Example project overview
   - Deployment Settings
   - Django ORM – Models, Queries and Migrations
   - Views – Functional and Class Based Views
   - URLs
   - Front-end and the Django Template Language
   - Admin
   - Forms
6. Summary

# Introduction

Course Requirements

# Course requirements

- Python
- An IDE / code editor
- The course contents
  - Code
  - PDF of slides

# Useful skills/knowledge

- Python skills:
  - Functions, lists, and dictionaries
  - Using external packages
  - Creating classes
  - Inheriting classes
  - Decorators
- Working with databases and SQL
- HTML and CSS
- HTTP requests

# Course setup

- Python 3.8 or higher installed (I'll be using 3.11)
  - Latest version: https://www.python.org/downloads/
- Course material downloaded and unzipped
  - https://github.com/ariannedee/intro-to-django
- An IDE for Python
  - I'll be using PyCharm Community edition (free)
  - For work, I use the Professional edition (30-day free trial)
  - VS Code is okay

# Install Python

https://www.python.org/downloads/

# Course material

https://github.com/ariannedee/intro-to-django

# Get an IDE for Python

PyCharm - Community Edition

https://www.jetbrains.com/pycharm/download/

# Open the course code

In your IDE, open the folder:

- **intro-to-django** if you cloned the code
- **intro-to-django-main** if you downloaded a zip file

# Folder structure

- intro-to-django/
  - dj_test_1/
  - dj_test_2/
  - docs/
  - sample_files/
  - trivia_site/
  - .gitignore
  - README.md
  - requirements.txt

# Folder structure

- intro-to-django/
  - **dj_test_1/** Example code solutions for Lesson 3
  - **dj_test_2/** Example code solutions for Lesson 4
  - docs/
  - sample_files/
  - trivia_site/
  - .gitignore
  - README.md
  - requirements.txt

# Folder structure

- intro-to-django/
  - dj_test_1/
  - dj_test_2/
  - docs/
  - **sample_files/** Files you'll need for Lessons 3 & 4
  - trivia_site/
  - .gitignore
  - README.md
  - requirements.txt

# Folder structure

- `intro-to-django/`
  - `dj_test_1/`
  - `dj_test_2/`
  - `docs/`
  - `sample_files/`
  - **`trivia_site/`** Example project for Lesson 5
  - `.gitignore`
  - `README.md`
  - `requirements.txt`

# Introduction

## Overview of **Django**

The web framework for
perfectionists with deadlines.

# Meet Django

- Build features fast
- Security built-in
- Scales in size well
- Built in Python

https://www.djangoproject.com/

# Meet Django

- Web framework written in Python
- Full featured
  - Security
  - Admin
  - Sessions
  - User management
- Used by many big websites/web apps

https://www.djangoproject.com/

# Websites who use Django

- Instagram
- Spotify
- YouTube
- The Washington Post
- The New York Times
- The Guardian
- NASA Website
- DoorDash
- Udemy

# Python Web Framework Alternatives

- For simpler/single page websites
  - Consider **Flask**
- For API's
  - Consider **FastAPI**
- For when speed and developer resources are tight
  - Consider **Anvil**

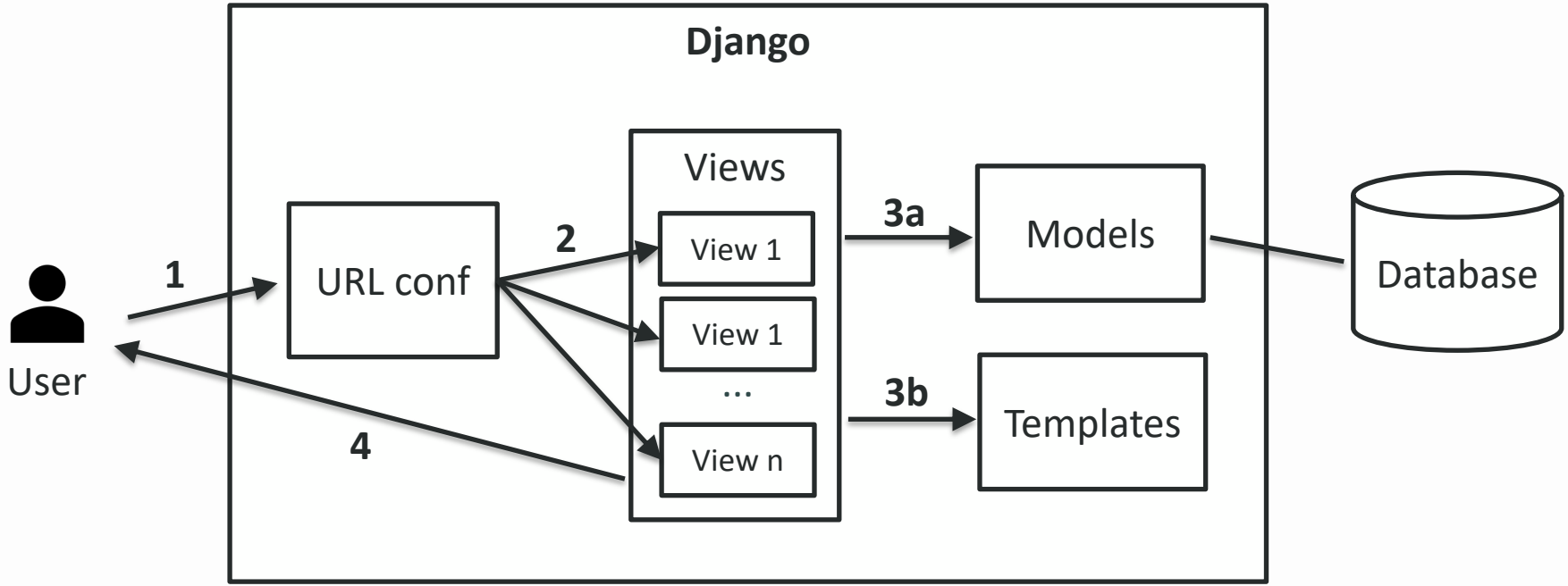
# History of Django

- Started in 2003 by Adrian Holovaty and Simon Willison (and later, Jacob Kaplan Moss) at a newspaper company in Kansas
- Released as open-source in 2005, and named after the jazz guitarist, Django Reinhardt
- Built to create things fast, under deadlines, with many non-developers, like content creators
- Read more at Django Design Patterns – The Story of Django

# Overall architecture

- Uses an MVT pattern:
  - **Models** – Interact with the database via an ORM
  - **Views** – Handle HTTP requests and return responses
  - **Templates** – Create dynamic HTML pages from Python data

# Django architecture overview



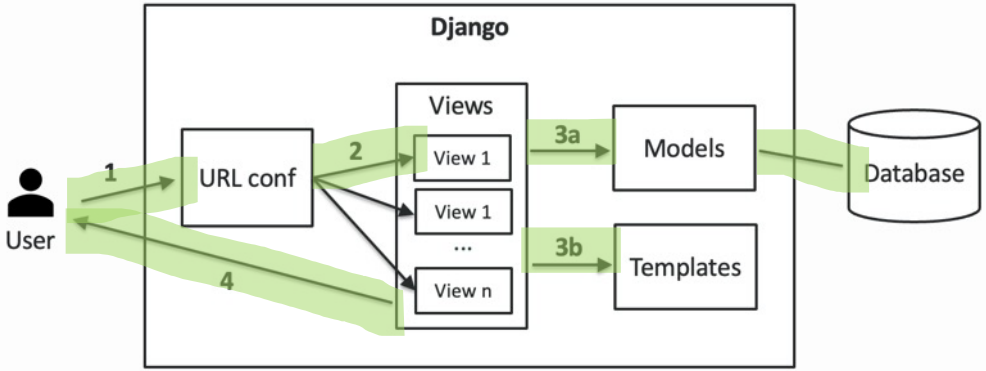
# Django architecture

1. User sends an HTTP **request** to Django

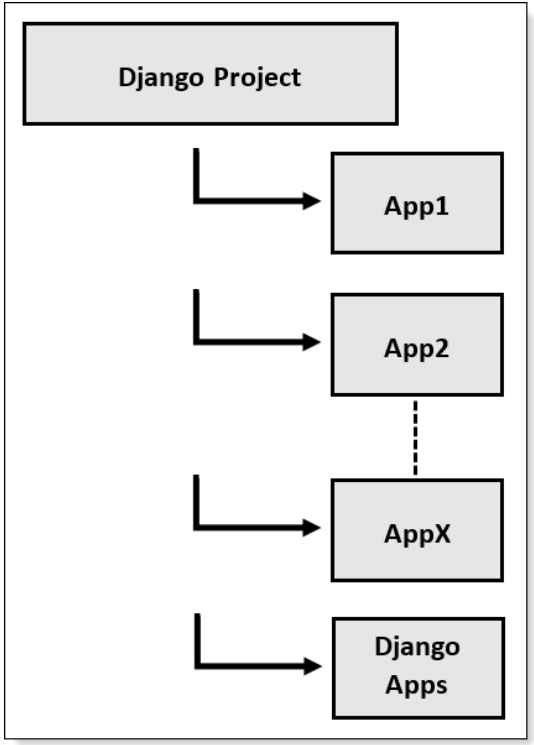
2. **URL conf**iguration, contained in urls.py, selects a View to handle the request

3. The **View** gets the request and
   a) Talks to a database via the **Models**
   b) Renders an HTML **Template**

4. The View returns an **HttpResponse** which gets sent to the client to be rendered as a web page in the browser



# Django project & apps



# Get Started With Django

# Get Started With Django

Learning resources

# Django resources

- Official Django site
  - https://www.djangoproject.com/
- Documentation
  - https://docs.djangoproject.com/
- Official tutorial
  - https://docs.djangoproject.com/en/4.2/intro/

# Official tutorial

- Django at a glance
- Quick install guide
- Writing your first Django app, part 1
- Writing your first Django app, part 2
- Writing your first Django app, part 3
- Writing your first Django app, part 4
- Writing your first Django app, part 5
- Writing your first Django app, part 6
- Writing your first Django app, part 7
- Advanced tutorial: How to write reusable apps
- What to read next
- Writing your first patch for Django

- https://docs.djangoproject.com/en/4.2/intro/
- First steps to understanding Django
- Parts 1-3 are essential for beginners

# Topic guides

- Managing files
  - Using files in models
  - The `File` object
  - File storage
- Testing in Django
  - Writing and running tests
  - Testing tools
  - Advanced testing topics
- User authentication in Django
  - Overview
  - Installation
  - Usage
- Django's cache framework
  - Setting up the cache
  - The per-site cache
  - The per-view cache

- https://docs.djangoproject.com/en/4.2/topics/
- If you've need help moving past the tutorial

# “How-to” guides

- Authentication using `REMOTE_USER`
- Writing custom `django-admin` commands
- Writing custom model fields
- Custom Lookups
- Custom template backend
- Custom template tags and filters
- Writing a custom storage system
- Deploying Django
- Upgrading Django to a newer version
- Error reporting
- Providing initial data for models
- Integrating Django with a legacy database
- Outputting CSV with Django
- Outputting PDFs with Django
- Overriding templates
- Managing static files (e.g. images, JavaScript, CSS)
- Deploying static files
- How to install Django on Windows
- Writing database migrations

- https://docs.djangoproject.com/en/4.2/howto/
- If you need help accomplishing a task

# API reference guides

- Databases
- `django-admin` and `manage.py`
- Running management commands from your code
- Django Exceptions
- File handling
- Forms
- Middleware
- Migration Operations
- Models
- Paginator
- Request and response objects

- https://docs.djangoproject.com/en/4.2/ref/

- If you want to know:
  - What a specific feature does
  - What the available features are
  - How to use a feature

# Beginner Django resources

- **A Wedge of Django** (book, previously Django Crash Course)
  - By Daniel and Audrey Feldroy
  - feldroy.com/products/a-wedge-of-Django

- **Build a website with Django 3** (book)
  - By Nigel George
  - https://djangobook.com/build-a-website-with-django-3/

- YouTube, online courses, and website tutorials

# Intermediate Django resources

- **Mastering Django** (book)
  - By Nigel George
  - https://djangobook.com/mastering-django-2-book/

- **Web Development in Python with Django** (video)
  - Building Backend Web Applications and APIs with Django
  - By Andrew Pinkham
  - https://learning.oreilly.com/videos/web-development-in/9780134659824/

# Get Started With Django

Before you start

# Before you start

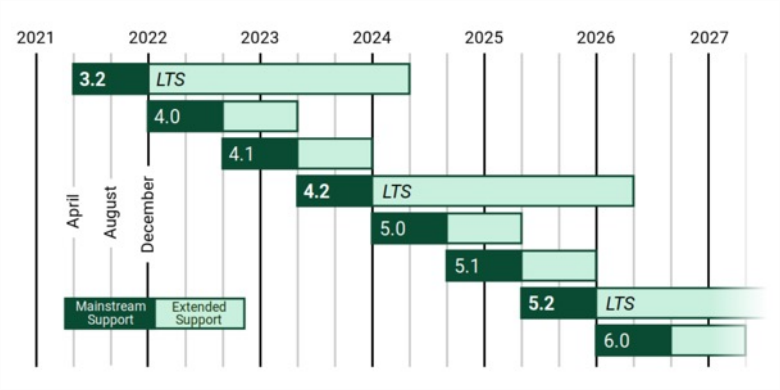
- Choose a Django version
- Choose a compatible Python version
- Create project folder
- Create a virtual environment with Python version

# Before you start

- **Choose a Django version**
- Choose a compatible Python version
- Create project folder
- Create a virtual environment with Python version

# Choose a Django version

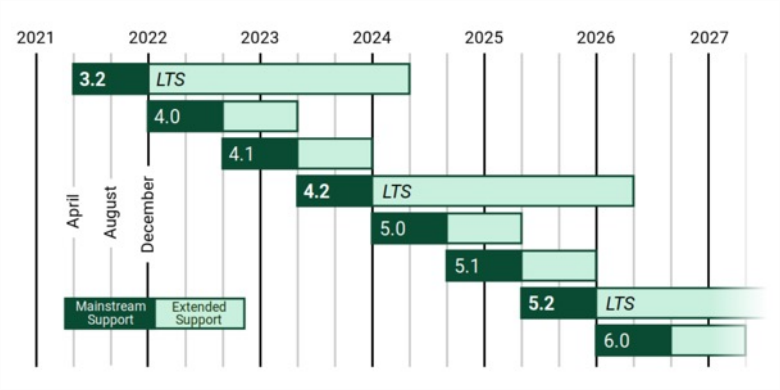
- Use an older version if you're running an existing project that hasn't been migrated yet
- Otherwise, use the latest X.2 release or the newest version
- X.2 will have long term support (LTS)




https://www.djangoproject.com/download/

# Choose a Django version

- 4 is very similar to 3
- 4 uses `zoneinfo` instead of `pytz`
- We're going to use 4.2
- Support for Async/ASGI started in 3.0 and is ongoing




https://www.djangoproject.com/download/

# Before you start

- Choose a Django version
  - Django 4.2
- **Choose a compatible Python version**
- Create project folder
- Create a virtual environment with Python version

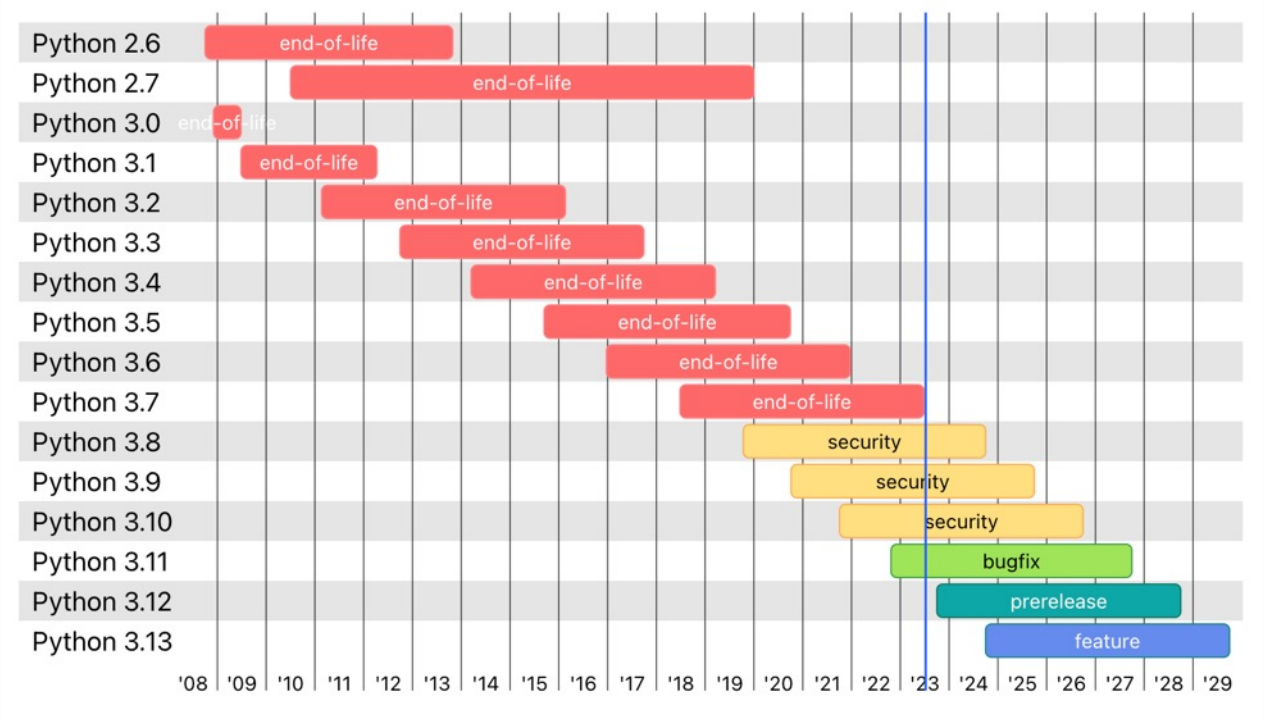
# Choose a Python version

## What Python version can I use with Django?

| Django version | Python versions |
|---|---|
| 3.2 | 3.6, 3.7, 3.8, 3.9, 3.10 (added in 3.2.9) |
| 4.0 | 3.8, 3.9, 3.10 |
| 4.1 | 3.8, 3.9, 3.10, 3.11 (added in 4.1.3) |
| 4.2 | 3.8, 3.9, 3.10, 3.11 |

https://docs.djangoproject.com/en/4.2/faq/install/

# Python Release Cycle




https://devguide.python.org/versions/

# Before you start

- Choose a Django version
  - Django 4.2
- Choose a compatible Python version
  - Python 3.11
- **Create project folder**
  - **intro-to-django** folder that you got from GitHub
- Create a virtual environment with Python version

# Before you start

- Choose a Django version
  - Django 4.2
- Choose a compatible Python version
  - Python 3.11
- Create project folder
  - **intro-to-django** folder that you got from GitHub
- **Create a virtual environment with Python version**
  - Optional for this course

# With PyCharm

- Open up `intro-to-django/` in PyCharm

- It may ask if you want to create a Virtual Environment
  - Choose the location `intro-to-django/venv`
  - Use `virtualenv`
  - Choose the Python version you decided on (Python 3.11)
  - Install requirements from `requirements.txt`

# Create a virtual environment with Python 3.11

Example for **Linux/Mac**:

```
$ cd Documents/intro-to-django
```

- Navigate into project folder

```
$ python3.11 -m venv venv
```

- Create a new virtual environment

```
$ source venv/bin/activate
```

- Activate virtual environment

# Create a virtual environment with Python 3.11

Example for **Windows**:

**`$ cd Documents/intro-to-django`**

- Navigate into project folder

**`$ python3.11 -m venv venv`** or **`py -m venv venv`**

- Create a new virtual environment

**`$ venv/bin/Activate.ps`**

- Activate virtual environment

# Why create a virtual environment?

- You may have multiple Python versions on your computer
- You may have projects that require different versions of the same external package
- Virtual environments create a "workspace" environment for a specific project
  - All installed apps are in one place, without conflicts
  - The "python" and "pip" commands point to the same place
  - Keeps your environment clean

# Connect it to your IDE

- Instructions for:
  - PyCharm
  - Visual Studio Code

- In PyCharm, mark your **venv** folder as **Excluded**

# Create a virtual environment

Unfamiliar with **`venv`** or the command line?

**Next Level Python LiveLessons**

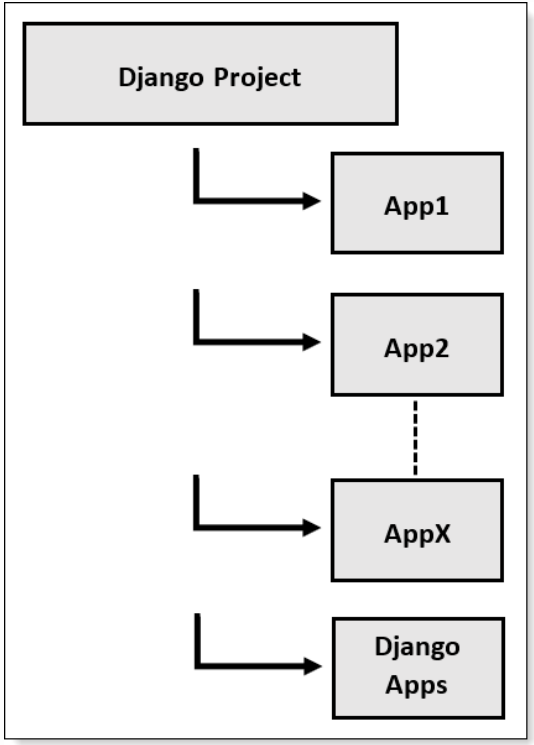
Lesson 3.3 - Create virtual environments using *venv*

Lesson 3.1 - Work with the command line

# Get Started With Django

Create a project

# Django project & apps



# Create a Django project

1. Install Django using pip
2. Create a new project using `django-admin startproject`
3. Run the development server
4. Initialize the development database
5. Create a superuser
6. View the admin site

# Create a Django project

1. **Install Django using pip**
2. Create a new project using `django-admin startproject`
3. Run the development server
4. Initialize the development database
5. Create a superuser
6. View the admin site

# Install Django using pip

In your activated virtual environment, use one of the following:

```
$ pip install django
```

Latest version of Django (any version)

```
$ pip install "django>=4.2,<5”
```

Latest version of Django 4.2

```
$ pip install "django==4.2.3”
```

Specific version of Django

```
$ pip install –r requirements.txt
```

Install from the *requirements.txt* file

# Install Django using pip

More instructions for installing packages using pip

**Next Level Python LiveLessons**

Lesson 3.2 - Install external libraries using pip

https://learning.oreilly.com/videos/next-level-python/9780136904083/9780136904083-NLP1_01_03_02/

# Create a Django project

1. Install Django using pip
2. **Create a new project using** `django-admin startproject`
3. Run the development server
4. Initialize the development database
5. Create a superuser
6. View the admin site

# Create a new Django project

```
$ django-admin startproject <project_name>
```

This will create a new folder with *project_name* and the following structure (if project name is **dj_test**)

- `dj_test`
  - `manage.py`
  - `dj_test`
    - `__init__.py`
    - `settings.py`
    - `urls.py`
    - `wsgi.py`
    - `asgi.py` (only in Django 3+)

# Create a new Django project

- `test_site`
  - **`manage.py`** ← This is the file you'll be running for most tasks
  - `test_site`
    - `__init__.py`
    - `settings.py`
    - `urls.py`
    - `wsgi.py`
    - `asgi.py` (only in Django 3+)

Navigate into the first **dj_test** folder in your terminal

```
$ cd dj_test
```

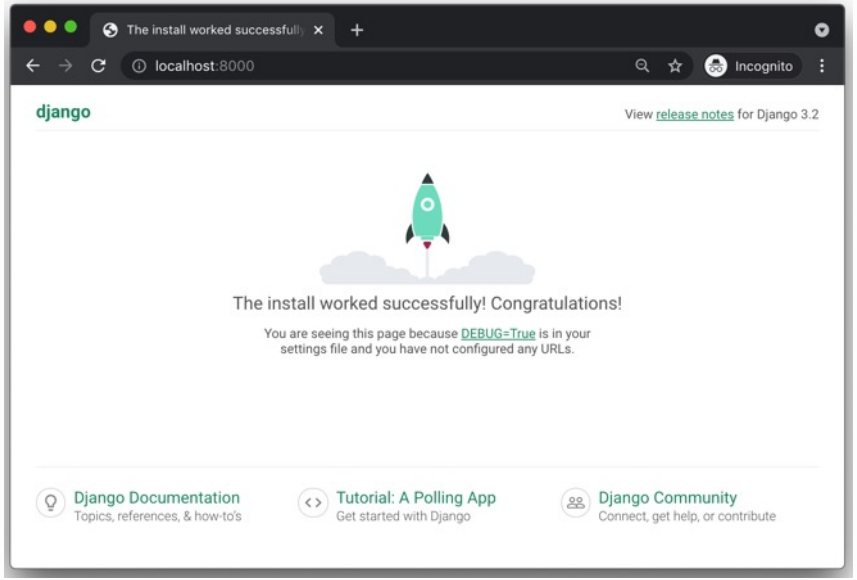
# Create a Django project

1. Install Django using pip
2. Create a new project using `django-admin startproject`
3. **Run the development server**
4. Initialize the development database
5. Create a superuser
6. View the admin site

# Run the development server

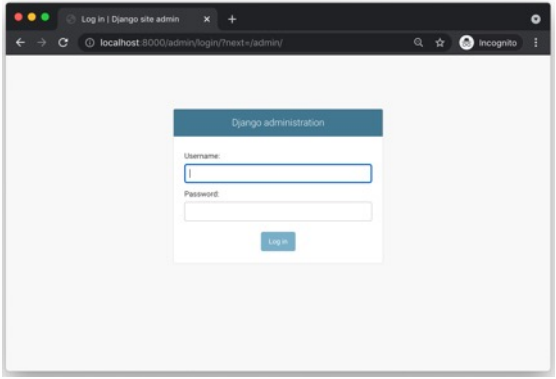
```
$ python manage.py runserver
```

Open http://127.0.0.1:8000/ or http://localhost:8000/ in your browser

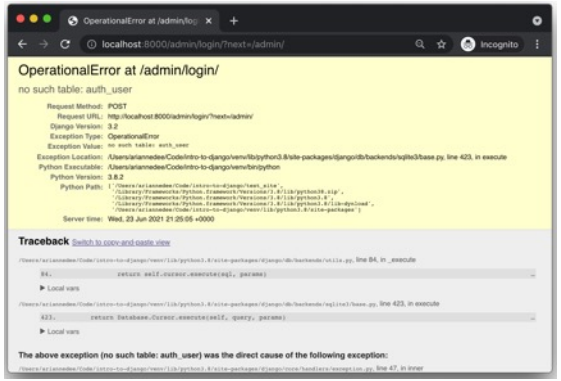


# Run the development server

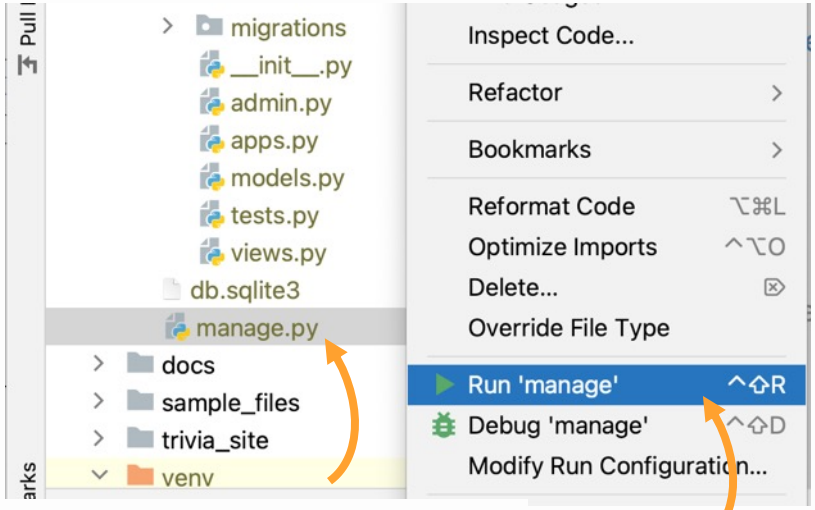
Now open
http://127.0.0.1:8000/admin




If you try to log in with any username/password, you'll get a server error



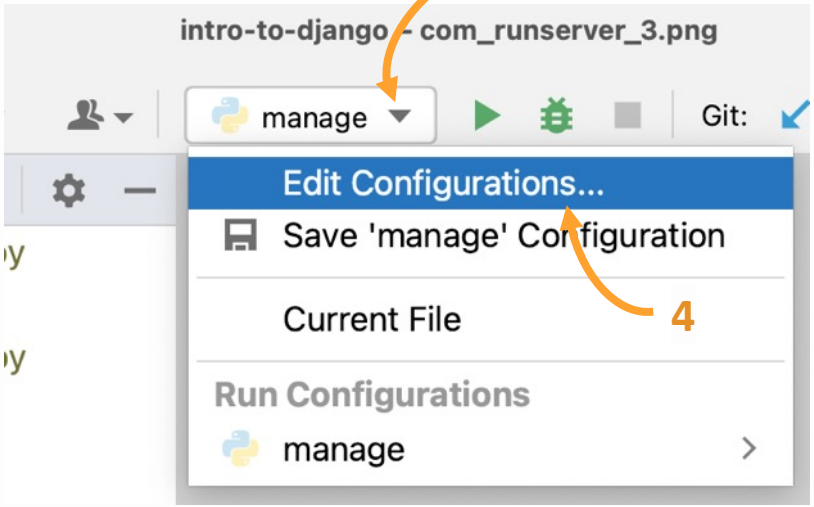
# Run the server through PyCharm




1. Right click on manage.py

2

3




4

# Run the server through PyCharm

Run/Debug Configurations

Python
Runserver

Name: Runserver
Allow multiple instances
Store as project file

Configuration Logs

Script path: /Users/ariannedee/Code/intro-to-django/dj_test/manage.py

Parameters: runserver

Environment

Environment variables:

PYTHONUNBUFFERED=1

Python interpreter: Project Default (Python 3.11 (intro-to-django)) ~/Code/intro-to-

Interpreter options:

Working directory: /Users/ariannedee/Code/intro-to-django/dj_test

Add content roots to PYTHONPATH

Add source roots to PYTHONPATH

Execution

Edit configuration templates...

Cancel Apply OK

**5. Add “runserver” to Parameters**

**6**

Runserver Git:

Run 'Runserver' ^R

# Create a Django project

1. Install Django using pip
2. Create a new project using `django-admin startproject`
3. Run the development server
4. **Initialize the development database**
5. Create a superuser
6. View the admin site

# Initialize the development database

```
$ python manage.py migrate
```

This will initialize a SQLite database (**db.sqlite3**) with the tables specified in the default project:

- authorization tables (user, permissions, groups)
- django_migrations
- django_admin_log
- django_sessions
- django_content_type

# Initialize the development database

You can edit the database type and default tables by editing the **`settings.py`** file

- **django_migrations**
  - required for Django's ORM feature
- **authorization tables** (user, permissions, groups)
  - default user and permissions features
- **django_admin_log**
  - tracks changes made via the Django Admin
- **django_sessions**
  - used in the sessions app (e.g. cookies)
- **django_content_type**
  - used for generic model relations (like tags, comments, likes, etc)

# View the data in the database

- **Command line interface**
- **An app**
  - e.g. DB Browser for SQLite or SQLiteStudio
- **A website**
  - e.g. https://inloop.github.io/sqlite-viewer/
- **PyCharm Professional** (built-in)
- **PyCharm Community**
  - plugin: Database Navigator
- **VS Code extension**
  - plugin: SQLite

# Create a Django project

1. Install Django using pip
2. Create a new project using `django-admin startproject`
3. Run the development server
4. Initialize the development database
5. **Create a superuser**
6. View the admin site

# Create a superuser

```
$ python manage.py createsuperuser
```

Creates your first user so that you can log into the admin site

You supply:

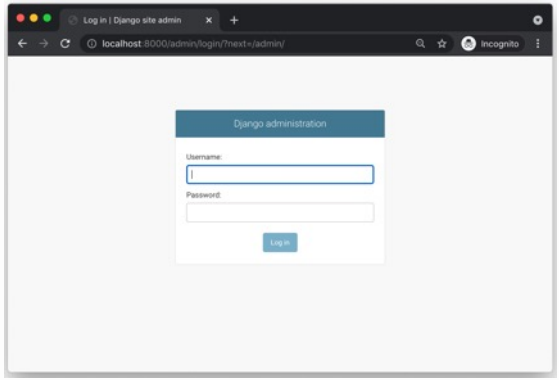
- username
- email address
- password
  - You can bypass the password requirements

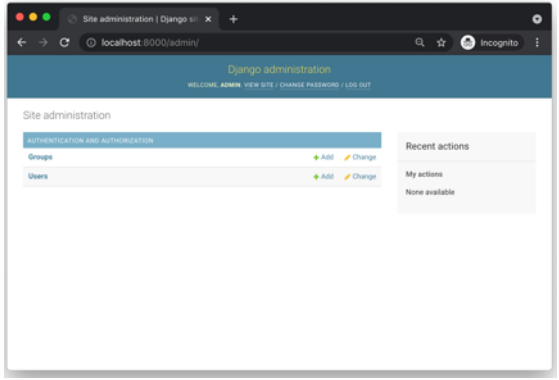
# Create a Django project

1. Install Django using pip
2. Create a new project using `django-admin startproject`
3. Run the development server
4. Initialize the development database
5. Create a superuser
6. **View the admin site**

# View the admin site

- Make sure your server is running
  - `$ python manage.py runserver`
- Open http://127.0.0.1:8000/admin
- Log in using your superuser credentials
- You can now browse, create, edit, and delete your Django models
  - only users and groups are configured initially
  - We'll add more later



# Get Started With Django

Default project files

# Initial project structure

- `dj_test`
  - `manage.py`
  - `dj_test`
    - `__init__.py`
    - `settings.py`
    - `urls.py`
    - `wsgi.py`
    - `asgi.py` (only in Django 3+)

Let's look at each of these files in order

# Root dj_test folder

- **`dj_test`** ← Root folder
  - `manage.py`
  - `dj_test`
    - `__init__.py`
    - `...`

# manage.py

- Run this file to run the project
  - **runserver** – Run the development server
  - **migrate** – Apply the migrations to the db
  - **createsuperuser** – Create a super user in the db
  - **shell** – Run a python console (has access to Django project)
  - **makemigrations** – If models changed, create migration file

- Run the file without any arguments to see full list of commands

Docs - https://docs.djangoproject.com/en/4.2/ref/django-admin/

# Custom manage.py commands

- You can create custom management commands

Docs

- https://docs.djangoproject.com/en/4.2/howto/custom-management-commands/

Tutorial

- https://simpleisbetterthancomplex.com/tutorial/2018/08/27/how-to-create-custom-django-management-commands.html

# dj_test project folder

- `dj_test`
  - `manage.py`
  - **`dj_test`** ← Be careful if you want to rename this
    - `__init__.py`
    - `...`

- This is where the overall configuration of your site goes
- Find all "**`dj_test`**" in project to see where it's being used

# settings.py

- Configures Django settings for your site

- Some notable ones:
  - DEBUG mode (must turn off in production)
  - Database settings (don't use SQLite in production)
  - Installed apps
  - Timezone
  - Static and media folder locations
  - Password validators

- We'll discuss settings more in the next section

Docs - https://docs.djangoproject.com/en/4.2/topics/settings/

# urls.py

Defines the URLs for your site

- For example, in our trivia app:
  - /  # index
  - /trivia/question/5/view  # view question with id=5
  - /admin/auth/user/1/change/  # change the user with id=1

- Map the URLs to templates, views or other URL sets (confs)

Docs - https://docs.djangoproject.com/en/4.2/topics/http/urls/

# urls.py

- In the default `urls.py`

```
urlpatterns = [
    path('admin/', admin.site.urls),
]
```

- The only URL pattern defined is linking the "admin/" path to the set of admin urls

★ Try changing the "admin/" path. How does that change the URLs in our existing test project?

# wsgi.py and asgi.py

- **WSGI** - Web Server Gateway Interface

- During development, you run your web server locally through
  - `manage.py runserver`

- In production, this is insufficient because it:
  - Loads static files slowly
  - Only handles one user at a time
  - Has not been tested for security

- Use WSGI when running in production

# wsgi.py

- WSGI provides an interface for running your Django application in production with a dedicated web server (like nginx or Apache)
- You don't need to know much about web servers and application servers
- Just follow tutorials for setting up Django in production and you might end up needing this file

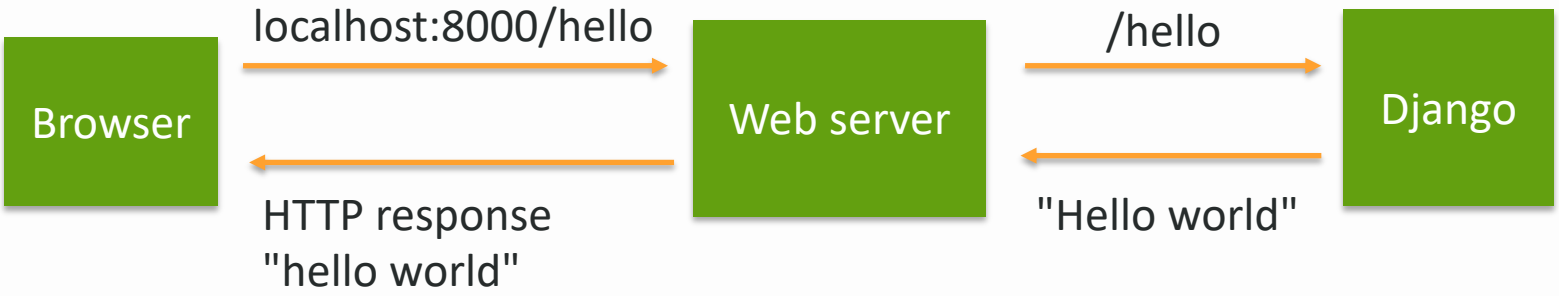
# asgi.py

- **ASGI** – Asynchronous Server Gateway Interface
- New in Django 3.0
- Similar to WSGI but for asynchronous web servers
  - Allows support for websockets, async and HTTP/2
- Overview of WSGI and ASGI in Django

# Get Started With Django

Create a view

# Requests & Responses



# Create a simple view

- Create a function that returns an HttpResponse

```
from django.contrib import admin
from django.urls import path
from django.http import HttpResponse

def my_func(request):
    return HttpResponse('Hello, world')

urlpatterns = [
    path('admin/', admin.site.urls),
    path('hello', my_func),
]
```

# Create a simple view

- Create a function that returns an HttpResponse
- Create a new path that returns that function

```
from django.contrib import admin
from django.urls import path
from django.http import HttpResponse

def my_func(request):
    return HttpResponse('Hello, world')

urlpatterns = [
    path('admin/', admin.site.urls),
    path('hello', my_func),
]
```

# Create a simple view

- Create a function that returns an HttpResponse
- Create a new path that returns that function

```
from django.contrib import admin
from django.urls import path
from django.http import HttpResponse

def my_func(request):
    return HttpResponse('Hello, world')

urlpatterns = [
    path('admin/', admin.site.urls),
    path('hello', my_func),
]
```

Don't call the function!

# Create a simple view

- Try returning some HTML in the HttpResponse

```
def my_func(request):
    return HttpResponse('<h1>Hello</h1><p>world<p>')
```

- You'll see how to return HTML from **templates** later

# Create a simple view

- If we put all of our logic in **urls.py**
  - Very big
  - Hard to separate functionality
  - Can't reuse features in other projects

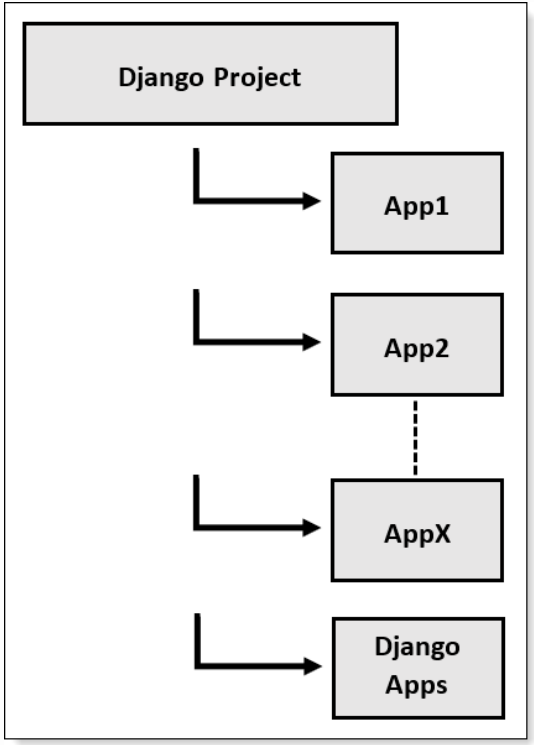
- Next we'll look at how to do the same thing in a dedicated **app**

# Write a Django App

# Write a Django App

Overview of Django apps

# Django project & apps



# Possible apps

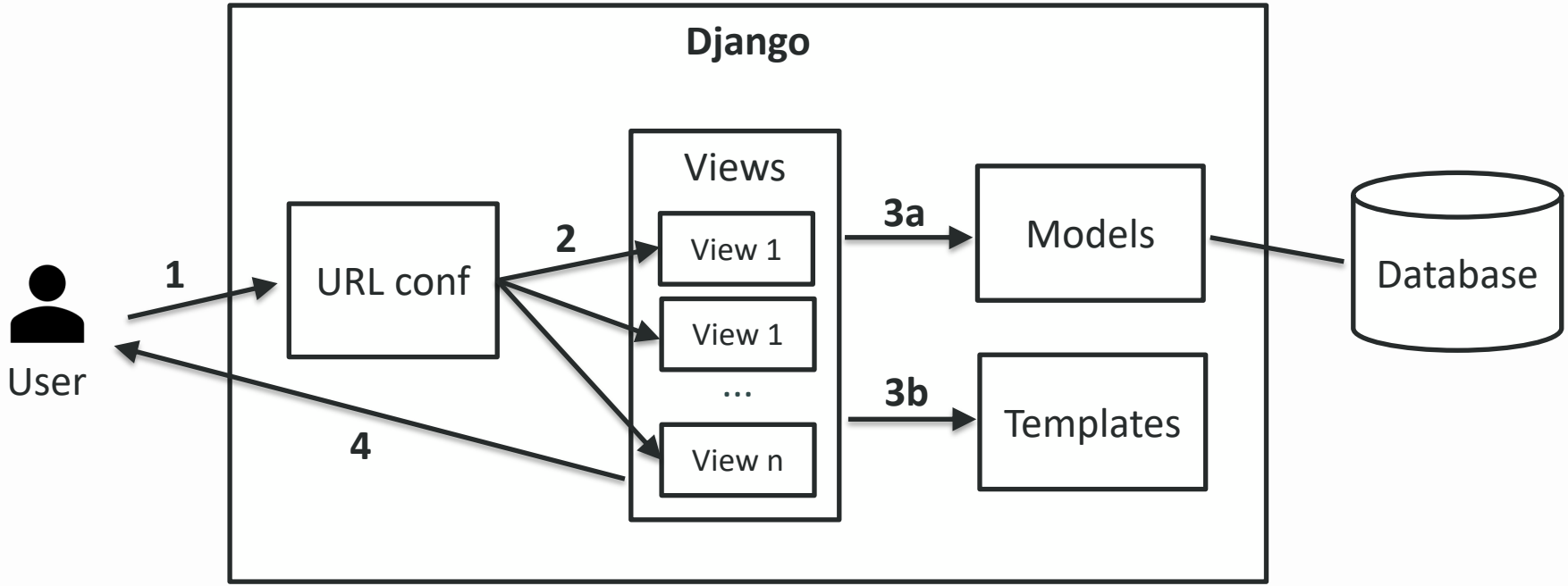
- General
  - Users
  - Blog
  - Events
  - News

- Trivia site specific
  - Questions
  - Categories
  - Leaderboard
  - Tournaments

# Django apps

- Admin
- Authorization
- Sessions
- Messages
- + more

# Django architecture



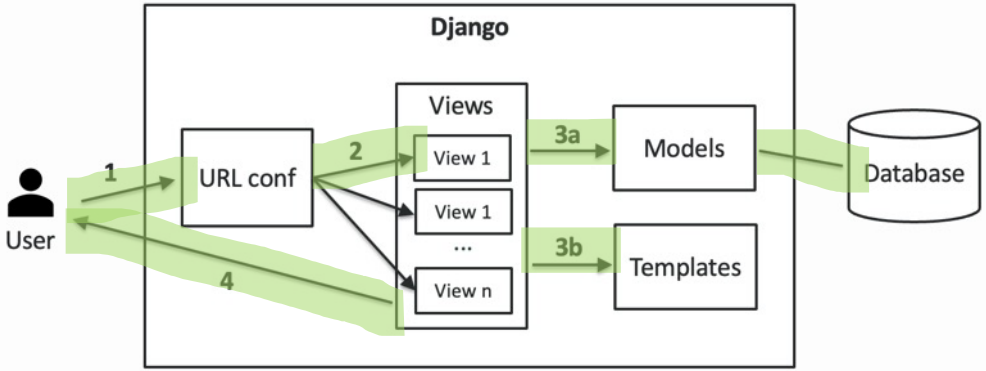
# Django architecture

1. User sends an HTTP **request** to Django

2. **URL conf**iguration, contained in urls.py, selects a View to handle the request

3. The **View** gets the request and
   a) Talks to a database via the **Models**
   b) Renders an HTML **Template**

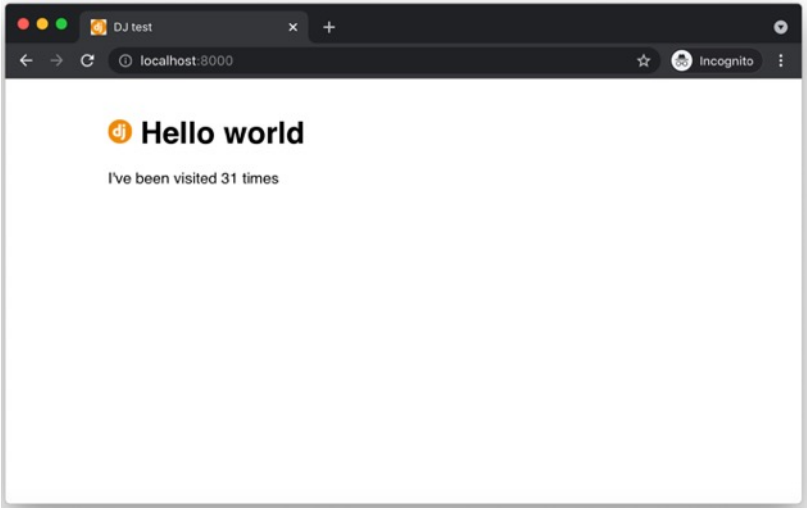
4. The View returns an **HttpResponse** which gets sent to the client to be rendered as a web page in the browser



# Write a Django App

Write a Visits app

# Our simple app




- Visits app
- Tracks how many times your site has been visited

# Create a Django app

1. Create a new app using `startapp`
2. Add it to the settings
3. Create a URL
4. Link it to a view
5. Return a template
6. Make it dynamic with context
7. Use data from a model
8. Add some static files

# Create a Django app

1. **Create a new app using `startapp`**
2. Add it to the settings
3. Create a URL
4. Link it to a view
5. Return a template
6. Make it dynamic with context
7. Use data from a model
8. Add some static files

# Create a new app using **startapp**

```
$ manage.py startapp visits
```

or

```
$ django-admin startapp visits
```

- visits
  - __init__.py
  - admin.py
  - apps.py
  - models.py
  - tests.py
  - views.py
  - migrations
    - __init__.py

# Create a Django app

1. Create a new app using `startapp`
2. **Add it to the settings**
3. Create a URL
4. Link it to a view
5. Return a template
6. Make it dynamic with context
7. Use data from a model
8. Add some static files

# Add it to the settings

- In **settings.py:**

```
INSTALLED_APPS = [
    'visits.apps.VisitsConfig',
    'django.contrib.admin’,
    ...
]
```

- Django uses **INSTALLED_APPS** as a list of places to look for models, management commands, tests, and other utilities

# Create a Django app

1. Create a new app using `startapp`
2. Add it to the settings
3. **Create a URL**
4. Link it to a view
5. Return a template
6. Make it dynamic with context
7. Use data from a model
8. Add some static files

# Create a URL – pt. 1

- Create a new file called **`urls.py`**

```
from django.urls import path
from . import views

app_name = 'visits'
urlpatterns = [
    path('', views.index, name='index'),
]
```

# Create a URL – pt. 1

- Create a new file called **`urls.py`**

```
from django.urls import path
from . import views
```

Relative import

```
app_name = 'visits'
urlpatterns = [
    path('', views.index, name='index'),
]
```

# Create a URL – pt. 2

- Connect it to **dj_test.urls.py**

```
from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path('admin/', admin.site.urls),
    path('', include('visits.urls'))
]
```

# Create a Django app

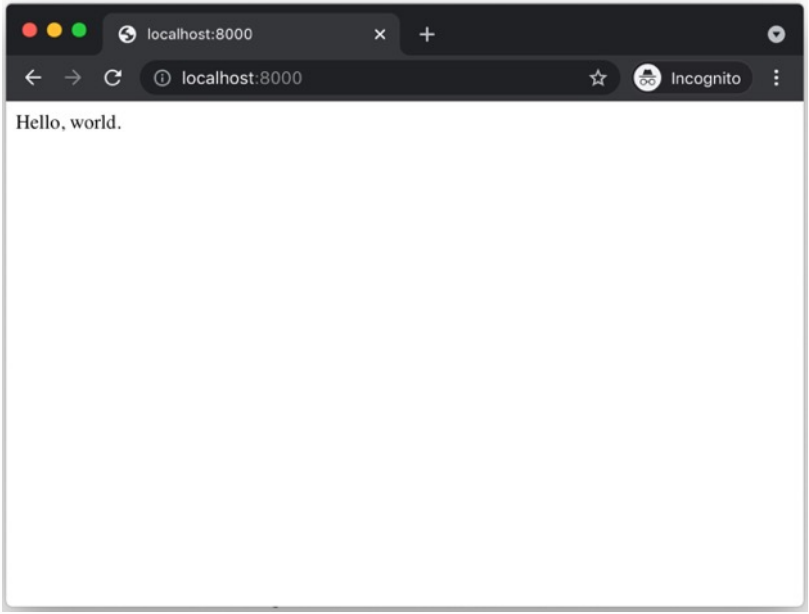
1. Create a new app using `startapp`
2. Add it to the settings
3. Create a URL
4. **Link it to a view**
5. Return a template
6. Make it dynamic with context
7. Use data from a model
8. Add some static files

# Link it to a view

- In **`views.py`**

```
from django.http import HttpResponse

def index(request):
    return HttpResponse("Hello, world!")
```

# Link it to a view




- We now have a simple index page

# Create a Django app

1. Create a new app using `startapp`
2. Add it to the settings
3. Create a URL
4. Link it to a view
5. **Return a template**
6. Make it dynamic with context
7. Use data from a model
8. Add some static files

# Return a template – pt. 1

- Create a new folder called **templates** in your **visits** folder
- Create a new file in **templates** called **index.html**
- Add some html (or copy it from *sample_files/index1.html*)

```html
<html>
  <head>
    <title>My Django Project</title>
  </head>
  <body>
    <h1>Hello world</h1>
    <p>I've been visited 1 time</p>
  </body>
</html>
```

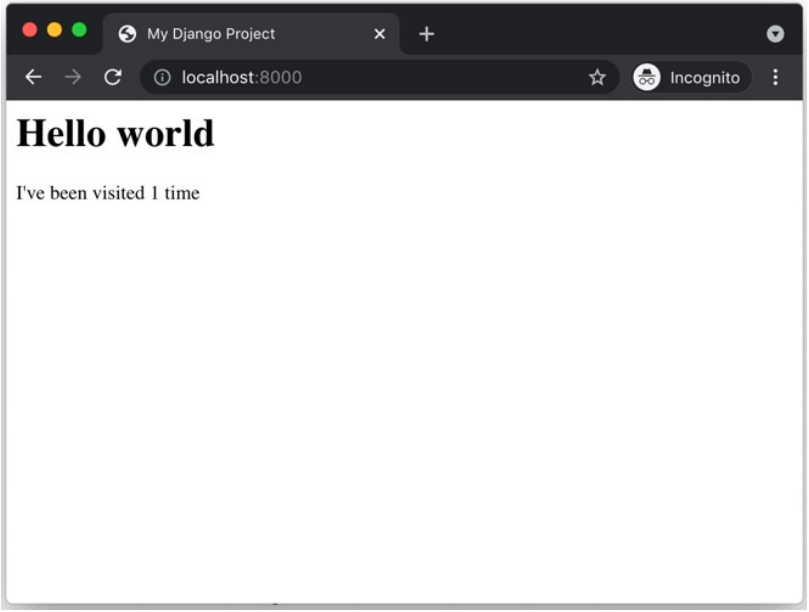
# Return a template – pt. 2

- Render the template from your view in **view.py**

```
from django.shortcuts import render

def index(request):
    return render(request, 'index.html')
```

- Render is a django "shortcut" that:
  1. Retrieves the template
  2. Renders it to an HTML file based on data that you pass in (none yet)
  3. Returns it as an HttpResponse

# Return a template




- This is just an HTML page until we start adding some dynamic content

# Create a Django app

1. Create a new app using `startapp`
2. Add it to the settings
3. Create a URL
4. Link it to a view
5. Return a template
6. **Make it dynamic with context**
7. Use data from a model
8. Add some static files

# Make it dynamic – pt. 1

- Send some data to the template from **views.py**

```
from random import randint
from django.shortcuts import render

def index(request):
    context = {"num_visits": randint(1, 5)}
    return render(request, 'index.html', context=context)
```

- Context is a dictionary that gets passed to the template
- The template can then use that data when rendering to HTML

# Make it dynamic – pt. 2

- Update your **index.html** template to use that data

```
<html>
  <head>
    <title>My Django Project</title>
  </head>
  <body>
    <h1>Hello world</h1>
    <p>
      I've been visited {{num_visits}} time{{num_visits|pluralize}}
    </p>
  </body>
</html>
```

# Make it dynamic – pt. 2

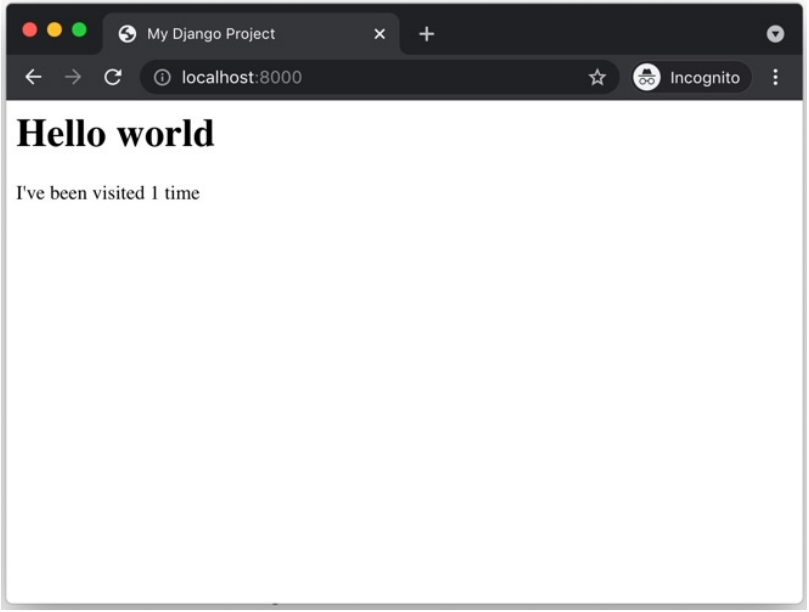
```
<body>
  <h1>Hello world</h1>
  <p>
    I've been visited {{num_visits}} time{{num_visits|pluralize}}
  </p>
</body>
```

## The Django Template Language

- The **{{ var }}** syntax is a **variable** that gets evaluated (from the context) and is replaced when the template is rendered

- The **{{ var|filter }}** syntax is a **filter**
- The **pluralize** filter returns an "**s**" if the variable is greater than 1

# Make it dynamic




- Every time you refresh the page, you will get a random number of visits
- "time" should be pluralized if num_visits is more than 1

# Create a Django app

1. Create a new app using `startapp`
2. Add it to the settings
3. Create a URL
4. Link it to a view
5. Return a template
6. Make it dynamic with context
7. **Use data from a model**
8. Add some static files

# Use data from a model – pt. 1

- In **models.py**, create a new model to store a visit to the site

```
from django.db import models

class Visits(models.Model):
    count = models.IntegerField(default=0)
```

# Use data from a model – pt. 2

- **Create a migration file**
  - `$ python manage.py makemigrations`
  - This will create a new file `0001_initial.py` in the migrations folder

- **Migrate the database**

  `$ python manage.py migrate`

# Use data from a model – pt. 3

- Create your initial **Visits** object
  - Open up a Django console in your terminal
    ```
    $ python manage.py shell
    ```
  - Import your Visits model
    ```
    >>> from visits.models import Visits
    ```
  - Create a Visits object
    ```
    >>> v = Visits.objects.create()
    ```
  - Check the current count value
    ```
    >>> v.count   # 0
    ```

# Use data from a model – pt. 4

- Update **views.py** so that
  - **visit.count** is updated every page view
  - it is sent to the context as **num_visits**

```
from django.shortcuts import render
from .models import Visits

def index(request):
    v = Visits.objects.first()
    v.count += 1
    v.save()
    context = {"num_visits": v.count}
    return render(request, 'index.html', context=context)
```

# Use data from a model – pt. 4

- Update **views.py** so that
  - **visit.count** is updated every page view
  - it is sent to the context as **num_visits**

```
from django.shortcuts import render
from .models import Visits

def index(request):
    v = Visits.objects.first()
    v.count += 1
    v.save()
    context = {"num_visits": v.count}
    return render(request, 'index.html', context=context)
```

Relative import

# More model options

1. Add a field to Visits that tracks the page that was viewed

2. Use a different Visit model that tracks individual page views
   - User who viewed (if logged in)
   - Date and time viewed
   - Page viewed

3. Keep track of user's IP address and location

# Create a Django app

1. Create a new app using `startapp`
2. Add it to the settings
3. Create a URL
4. Link it to a view
5. Return a template
6. Make it dynamic with context
7. Use data from a model
8. **Add some static files**

# Add some static files – pt. 1

- Create a new folder called **static** in your **visits** folder
- Copy **styles.css**, **logo.png**, and **favicon.ico** from ***sample_files*** to the new ***static*** folder

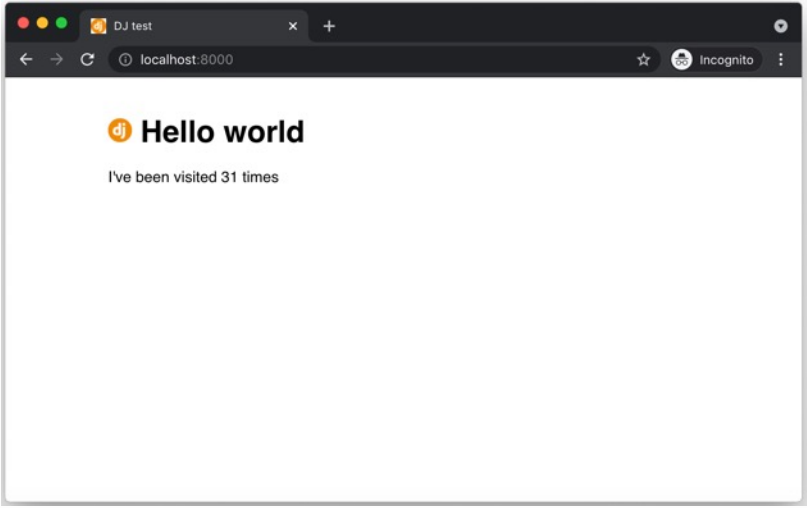
```
body {
  padding: 30px 100px;
  font-family: sans-serif;
}

img {
  width: 25px;
}
```

# Add some static files – pt. 2

- Update your **index.html**, or copy it from **sample_files/index2.html**

```
{% load static %}
<html>
<head>
  <title>DJ test</title>
  <link rel="icon" href="{% static 'favicon.ico' %}">
  <link rel="stylesheet" type="text/css" href="{% static 'styles.css' %}">
</head>
<body>
  <h1>
    <img src="{% static 'logo.png' %}"> Hello world
  </h1>
  <p>I've been visited {{ num_visits }} time{{ num_visits|pluralize }}</p>
</body>
</html>
```

# Add some static files




- **Restart** the development server and **refresh** your browser
- You should now see slightly nicer style, a logo in the header, and an icon in the tab

# Create a Django app

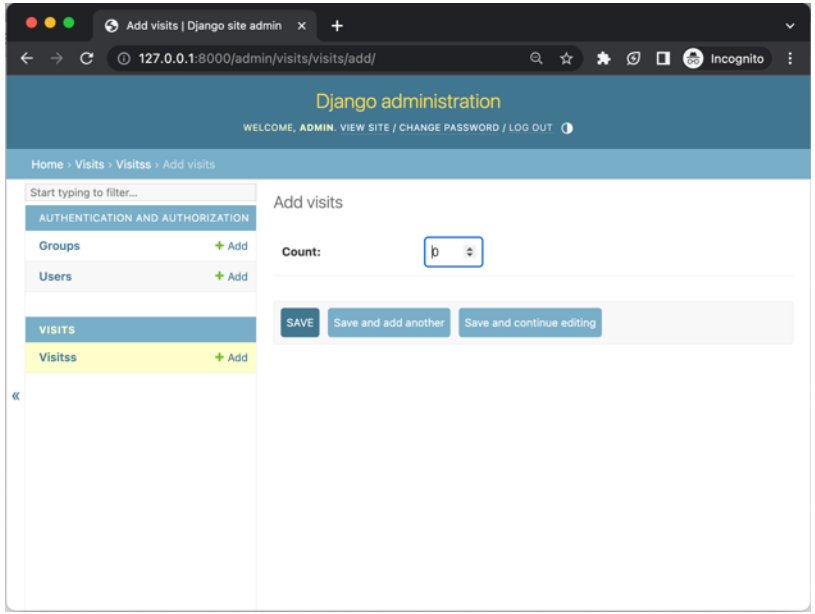
1. Create a new app using `startapp`
2. Add it to the settings
3. Create a URL
4. Link it to a view
5. Return a template
6. Make it dynamic with context
7. Use data from a model
8. Add some static files
9. **Edit data from the admin**

# Edit data from the admin – pt. 1

- Register the Visits model in **admin.py**

```
from django.contrib import admin
from .models import Visits

@admin.register(Visits)
class VisitsAdmin(admin.ModelAdmin):
    pass
```

# Edit data from the admin




- Open http://127.0.0.1:8000/admin
- Log in with your superuser
- Go to Visitss
- You can:
  - View the list of Visits
  - Edit a Visits
  - Add a Visits

# Basic Features

# Basic Concepts

Settings

# Settings

- Provides configuration for your Django project
- Default location is **settings.py**
- The Deployment Settings lesson discusses how to create different settings files for different environments
  - e.g. local vs production

# Settings

- Python file, python syntax
- Settings are constants
  - Use all uppercase
  - Reload server if any settings change
- Specify the location of your settings file
  - In manage.py
    - `os.environ.setdefault('DJANGO_SETTINGS_MODULE', 'dj_test.settings')`
  - Set environment variable **DJANGO_SETTINGS_MODULE**

# Specifying paths

- Django >= 3.1

```
from pathlib import Path
BASE_DIR = Path(__file__).resolve(strict=True).parent.parent.parent
STATICFILES_DIRS = [BASE_DIR / "static"]
```

- Django <= 3.0

```
import os
BASE_DIR = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
STATICFILES_DIRS = [os.path.join(BASE_DIR, 'static')]
```

# Must-know settings

- **DEBUG**
  - Show detailed error page vs 404.html or 500.html
  - Never set it to True in production
- **INSTALLED_APPS**
  - All Django, 3rd party and local apps used
- **DATABASES**
  - Can configure multiple databases
  - Should be different for local, production, staging

# Project structure

- **BASE_DIR**
  - Set to project root (where manage.py is located)
- **APP_DIR**
  - Set to where apps are located
- **TEMPLATES**
  - **DIRS** - Where to look for templates
  - **APP_DIRS** – If True, look for a "templates" folder in each app
- **STATICFILES_DIRS**
  - Where to look for static files
- **MEDIA_ROOT**
  - Where to look for media files (user-uploaded)

# Time zones

- **USE_TZ**
  - True if dates/times should be timezone-aware by default
  - Should set to True
- **TIME_ZONE** – Set it to your time zone

- **USE_I18N** – Enable text translation (set to False if not supported)
- **USE_L10N** – Format data based on locale

- Docs:
  - Time zones
  - Text translation
  - Date and time formatting

# Security

- AUTH_PASSWORD_VALIDATORS
- SESSION_EXPIRE_AT_BROWSER_CLOSE

# Basic Concepts

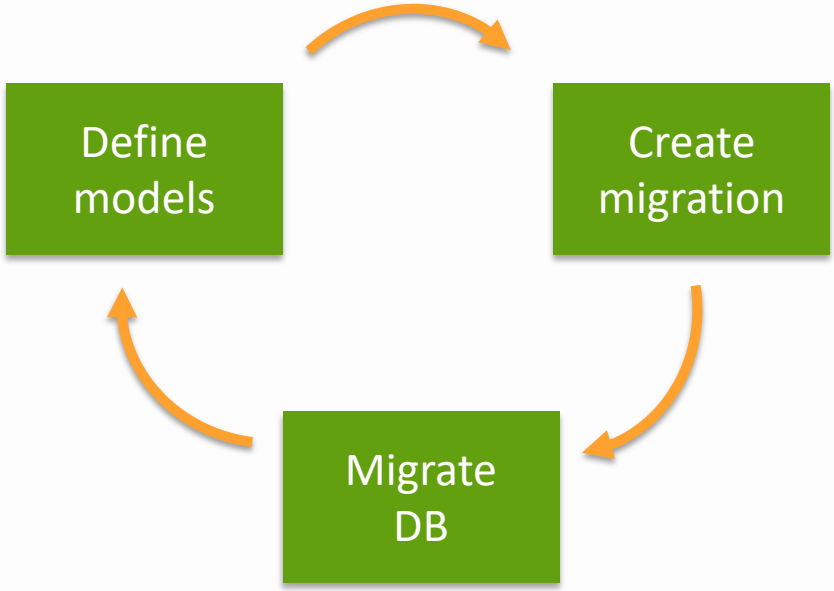
Django ORM

# The Django ORM

- **O**bject-**R**elational **M**apping
- Manage your data and database with Python instead of SQL
- Lazy evaluation of queries (database only gets accessed when data is requested, not when forming queries)
- Easy things are easy, medium things are possible, hard things can be done in SQL

# Schema Update Workflow

- Define models with fields and relationships
- Create a migration file
- Migrate the database



# Schema Update Workflow

- Define models with fields and relationships
- Create a migration file
  - `$ python manage.py makemigrations`
- Migrate the database
  - `$ python manage.py migrate`

# Update the Visits models

- Rename `models.Visits` to `Visit` so references get renamed

- Update `models.py`

```
class Visit(models.Model):
    page = models.CharField(max_length=255)
    username = models.CharField(max_length=255, default="anonymous")
    timestamp = models.DateTimeField(auto_now_add=True)
```

# Model fields

- Many different model fields available
- Some options are required
  - E.g. **CharField** requires **max_length**

- **DateTimeField** has some special options, like **auto_now_add**
  - Set to the current datetime when added to the db

- More fields and options discussed in the next lesson

# Migrate the database

- Generate a migrations file
  - `$ python manage.py makemigrations`
- Migrate the database
  - `$ python manage.py migrate`

# Object creation

- Create a new Visit

```
v = Visit(page="about")
v.username = "admin"
v.save()
```

- Create a new Visit and save it in one line

```
v = Visit.objects.create(page="about")
```

# Object retrieval

- Get all Visits:

```
Visit.objects.all()
```

- Get a single Visit:

```
v = Visit.objects.first()
v.page
```

- Filter Visits:

```
visits = Visit.objects.filter(page="about")
visits.count()
```

# Basic Concepts

Views

# Update view.py

```python
from django.shortcuts import render
from visits.models import Visit

def index(request):
    v = Visit(page="")
    if request.user.is_authenticated:
        v.username = request.user.username
    v.save()

    visitors = Visit.objects.filter(page="")
    context = {
        "num_visits": visitors.count()
    }
    return render(request, "index.html", context)
```

# The request object

- A view function always accepts an **HttpRequest** object
  - from django.http.request

- An **HttpRequest** has:
  - A user (as long as you're using the Django auth middleware)
    - Will be an anonymous user if they're not logged in
  - body – content of the request
  - GET/POST – query parameters
  - Headers – metadata
  - request URL

# View responses

- A view function always returns an **HttpResponse** object

- An **HttpResponse** has:
  - A status code (like 200 Okay)
  - Some content (like HTML or JSON)
  - Some metadata (in headers)

# Basic Concepts

URLs

# Add a URL pattern to urls.py

```
from django.urls import path

from . import views

app_name = 'visits'
urlpatterns = [
    path('', views.index, name='index'),
    path(‘<str:page>', views.index, name='index')
]
```

# Accepting arguments

- Use <> to capture URL arguments and pass them to the view

```
path('users/<username>/', views.profile)
```

- Arguments are strings by default, but you can specify the type

```
path('users/<int:pk>/, views.profile)
```

# Accept argument in views.py

```
from django.shortcuts import render
from visits.models import Visit

def index(request, page=""):
    v = Visit(page=page)
    if request.user.is_authenticated:
        v.username = request.user.username
    v.save()

    visitors = Visit.objects.filter(page=page)

    context = {
        "num_visits": visitors.count()
    }
    return render(request, "index.html", context)
```

# Basic Concepts

Templates

# Pass more context in views.py

```
context = {
    "page": page,
    "visitors": visitors,
    "num_visits": visitors.count(),
}
```

# Update the body of index.html

```
<body>
  <h1>
    <img src="{% static 'logo.png' %}"> {{ page|title }}
  </h1>
  <p>I've been visited {{ num_visits }} time{{ num_visits|pluralize }}</p>
  <ul>
    {% for visit in visitors %}
    <li>{{ visit.username }} at {{ visit.timestamp }}</li>
    {% endfor %}
  </ul>
</body>
```

# Templates

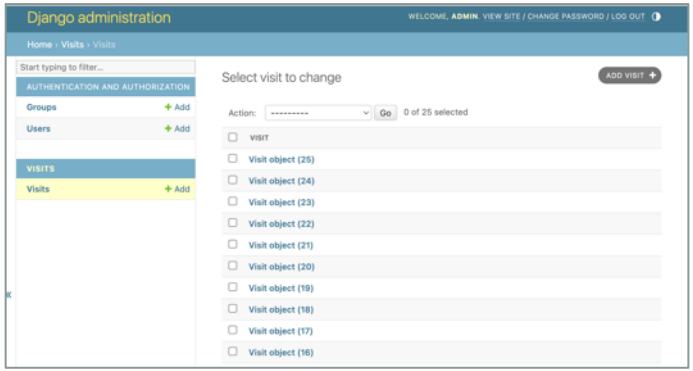
- Templates are just HTML files with special syntax that the Django Template Language (DTL) uses to replace with dynamic content

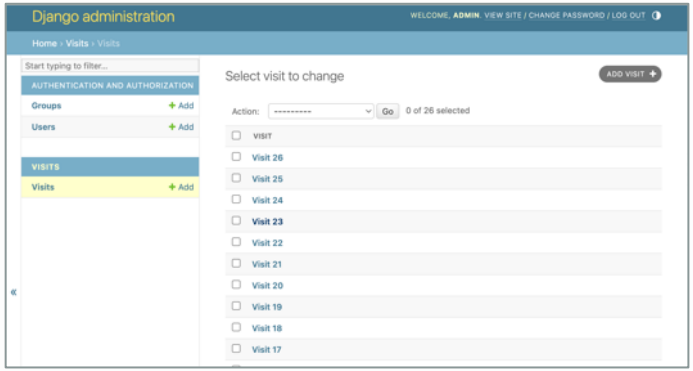
- Variable: `{{ var }}`
- Filter: `{{ var|filter }}`
- Tag: `{% tag %}`
- Comment: `{# comment #}`

# Basic Concepts

Django Admin

# List View



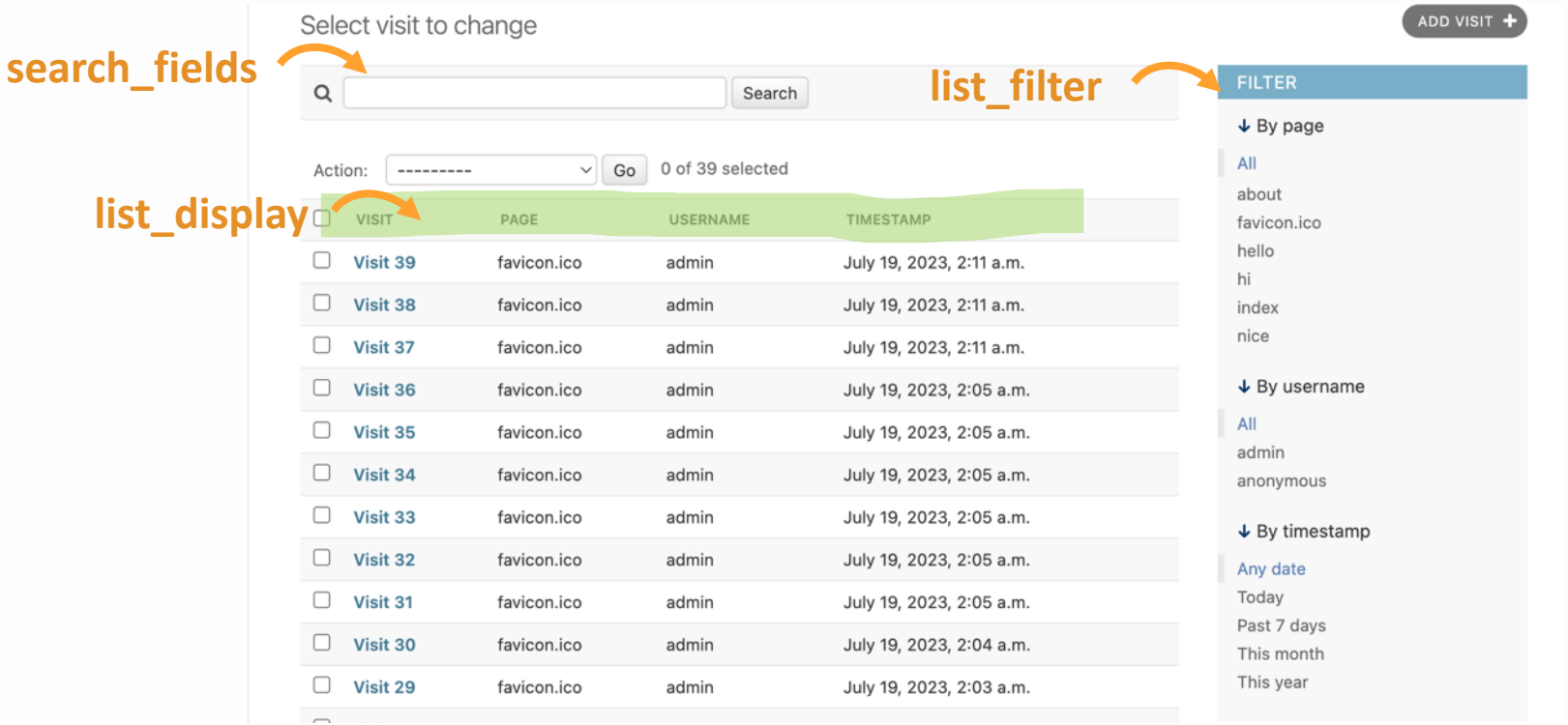

- Define a `__str__()` method on **Visit** to customize how it's displayed in the admin list view

```
def __str__(self):
    return f"Visit {self.pk}"
```

# Customizing the List View

```
from django.contrib import admin
from .models import Visit

@admin.register(Visit)
class VisitsAdmin(admin.ModelAdmin):
    list_display = ['__str__', 'page', 'username', 'timestamp’]
    list_filter = ['page', 'username', 'timestamp’]
    search_fields = ['page', 'username’]
    readonly_fields = ['timestamp']
```

# List View Updates



# Change view updates

Home › Visits › Visits › Visit 16

Change visit

HISTORY

**Visit 16**

Page: index

**Username:** anonymous

readonly_fields

Timestamp: July 19, 2023, 1:42 a.m.

SAVE | Save and add another | Save and continue editing | Delete

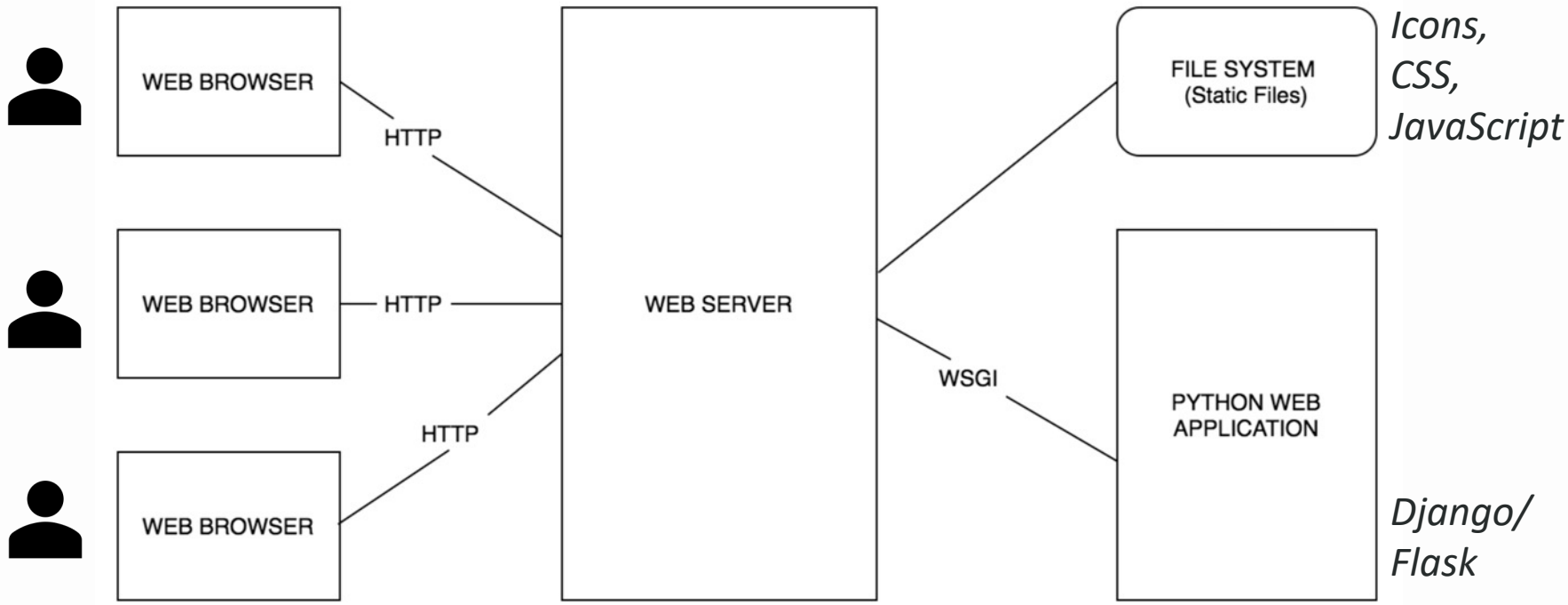
# Go forth and code!

# Intermediate Concepts

# Intermediate Concepts

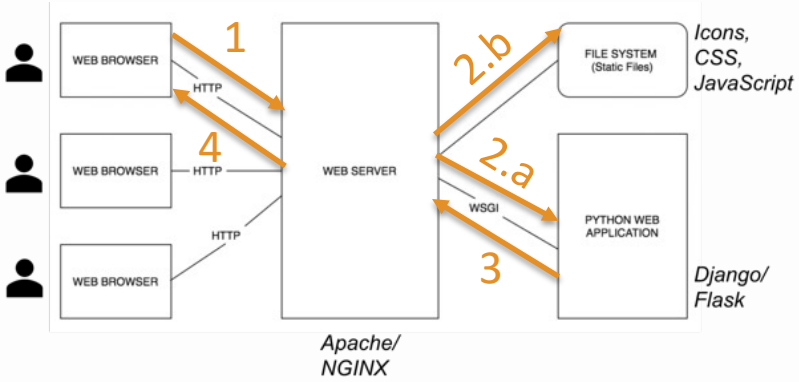
Django’s Architecture

# Web application overview




*Apache/*
*NGINX*

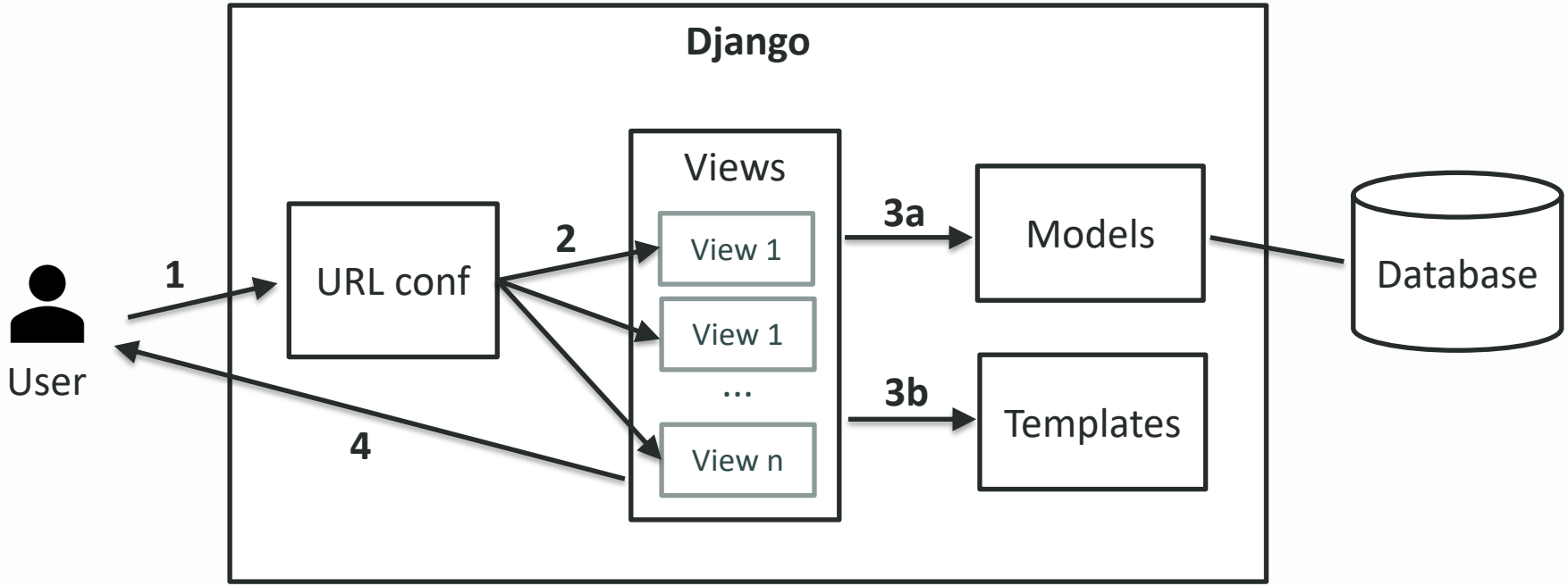
# Web application overview




1. Browser sends an **HTTP** request to your web server
2. Web server sends the request to
   a) a Python app server via WSGI
   b) the file system to get a file
3. The application returns an HTTP response to the web server
4. A rendered web page is seen in the browser

We mostly care about 2.a and 3

# Django architecture



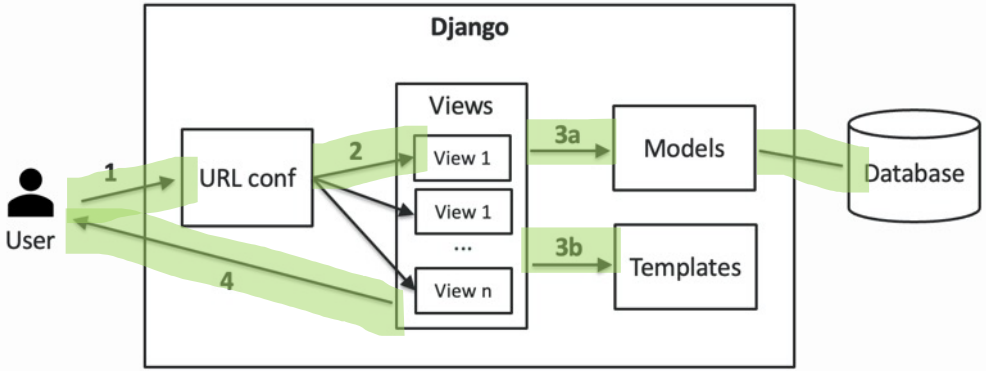
# Django architecture

1. User sends an HTTP **request** to Django

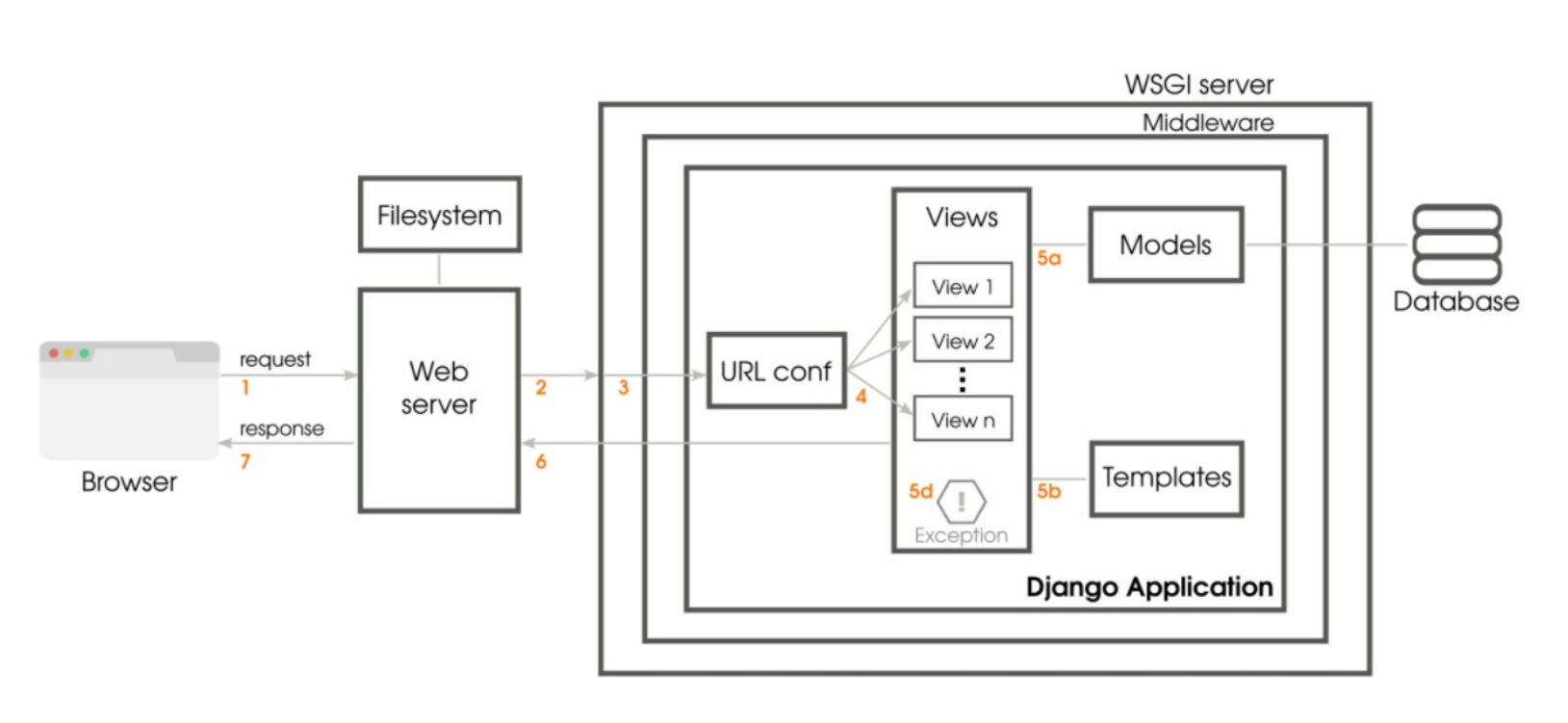
2. **URL conf**iguration, contained in urls.py, selects a View to handle the request

3. The **View** gets the request and
   a) Talks to a database via the **Models**
   b) Renders an HTML **Template**

4. The View returns an **HttpResponse** which gets sent to the client to be rendered as a web page in the browser



# Overall architecture



# Overall architecture

1. The browser sends the request (essentially, a string of bytes) to your web server.
2. Your web server (say, Nginx) hands over the request to a Web Server Gateway Interface (WSGI) server (say, uWSGI) or directly serves a file (say, a CSS file) from the filesystem.
3. Unlike a web server, WSGI servers can run Python applications. The request populates a Python dictionary called environ and, optionally, passes through several layers of middleware, ultimately reaching your Django application.
4. URLconf (URL configuration) module contained in the urls.py of your project selects a view to handle the request based on the requested URL. The request has turned into HttpRequest, a Python object.
5. The selected view typically does one or more of the following things:
   a) Talks to a database via the models
   b) Renders HTML or any other formatted response using templates
   c) Returns a plain text response (not shown)
   d) Raises an exception
6. The HttpResponse object gets rendered into a string, as it leaves the Django application.
7. A beautifully rendered web page is seen in your user's browser.

# Main tasks

- Configure **URL** structure
- Create **models** of our data
- Use models to manage the database
- Create HTML **templates** (which can use models and other data)

- Create **views** that can retrieve data from **models** and dynamically create an HTML page from a **template**
- Specify which **URLs** go to which **views**

- Define **settings** for different environments (local, production, etc.)

# Other Django features

- Built-in:
  - User authentication and permissions
  - Form creation
  - Admin interface
  - Many security features
  - Sessions/cookies
  - Signals

- Simple to set up:
  - Caching
  - Internationalization / localization
  - Debug toolbar

# Security features

- Cross-site scripting (XSS) protection
- Cross-site request forgery (CSRF) protection
- SQL injection protection
- Clickjacking protection
- SSL/HTTPS
- Host header validation

# When to use Django

- You want to work in Python
- You want to use its features, like the admin, ORM, authentication, sessions, etc
- You want to develop features quickly
- You need to worry about security and don't have the expertise/time to handle it from scratch
- You are okay working within the MVT architecture and don't need too much customization

# Popular Django apps/libraries

- **API creation**
  - Django REST Framework (DRF) – REST APIs
  - Graphene Django – GraphQL APIs
- **Debugging and performance testing**
  - Django Debug Toolbar
- **Scheduling and asynchronous tasks**
  - Celery
- **WebSocket support**
  - Channels

# Deployment options

- **Python Anywhere**
  - No WebSocket, ASGI, Celery queues, NoSQL, or Docker support
  - Postgres only on paid custom account
  - Slower to support newer Python and MySql versions

- **Heroku**
  - Can't serve media files (use Amazon S3)
  - Most expensive to scale
  - Less direct server control

- **Digital Ocean, Amazon Web Services and Microsoft Azure**
  - More technical, better cost and performance if you need scale

# Intermediate Concepts

Django project structures

# Starting a project

- project_folder/
  - venv/
  - `$ django-admin startproject dj_test`
  - dj_test/
    - manage.py
    - dj_test/
      - settings.py
      - urls.py
      - wsgi.py

# Start a project in the current directory

- project_folder/
  - venv/
  - `$ django-admin startproject dj_test .`
  - manage.py
  - dj_test/
    - settings.py
    - urls.py
    - wsgi.py

- Directory to put the project into.
- . is the current location.
- Defaults to project name (e.g. dj_test).

# Start a project in a default directory

- project_folder/
  - venv/
  - `$ django-admin startproject dj_test dj_test`
  - dj_test/
    - manage.py
    - dj_test/
      - settings.py
      - urls.py
      - wsgi.py

Default value

# Shallow project structure

- project_folder/
  - venv/
  - manage.py
  - dj_test/
    - settings.py
  - requirements.txt
  - README.md
  - .git/

# Nested project structure

- project_folder/
  - venv/
  - dj_test/
    - manage.py
    - dj_test/
      - settings.py
  - requirements.txt
  - README.md
  - .git/

# Alternative project structures

- Most people don't use the default project structure for their projects

- You can:
  - name the root folder and project config folder differently
  - put your apps in an **apps** folder
  - have a single location for templates and static files

- You can start projects using cookie-cutter

# Default project structure

- **dj_test**
  - manage.py
  - **dj_test**
    - settings.py
    - urls.py
  - **users**
    - static
    - templates
  - **visits**
    - static
    - templates

# Default project structure

- **dj_test** ← `django-admin startproject dj_test`
  - manage.py
  - **dj_test**
    - settings.py
    - urls.py
  - **users** ← `python manage.py startapp users`
    - static
    - templates
  - **visits** ← `python manage.py startapp visits`
    - static
    - templates

# Commonly used structure

- dj_test_root
  - manage.py
  - **dj_test_project**
    - settings.py
    - urls.py
  - apps
    - **users**
    - **visits**
  - static
  - templates
    - users
    - visits

# Cookiecutter

- Create new Django projects using Cookiecutter
  - Docs: https://cookiecutter.readthedocs.io/
  - Templates: https://github.com/search?q=cookiecutter+django

# cookiecutter-django structure

- dj_test
  - manage.py
  - **config**
    - urls.py
    - settings.py
  - dj_test
    - **users**
    - **visits**
    - static
    - templates
      - users
      - visits

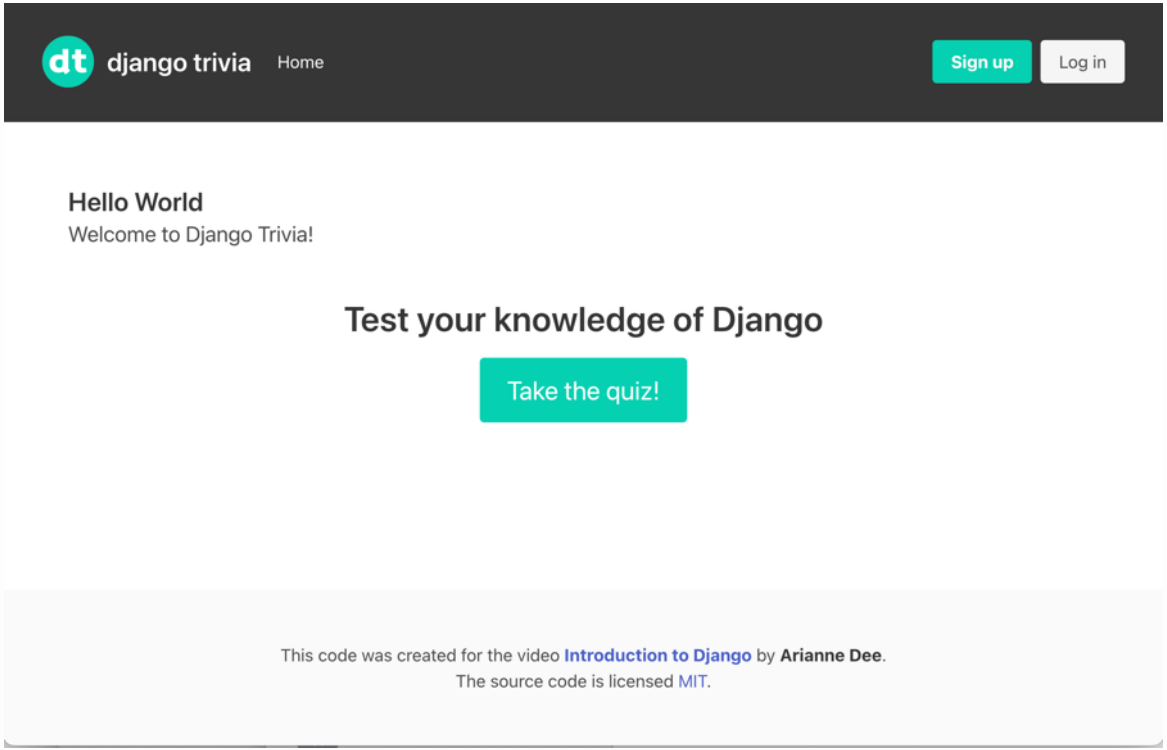
# Update settings for other structures

- In order to use other structures, you'll have to edit some settings and configurations
- Let **manage.py** know where the default setting file is
- In your settings file, add paths for Django to find your templates and static files
  - `TEMPLATES["DIRS"]`
  - `STATICFILES_DIRS`
- Your AppConfig's name in **apps.py** needs to use the full path
  - e.g. name = **'apps.users'**

# Intermediate Concepts

Example project overview

# Django trivia project




http://trivia-ariannedee.pythonanywhere.com/

# Features

- Basic user login/signup
- Staff can edit questions using Django forms
- Admin site for managing users and questions
- Sessions for remembering quiz state
- Production and local settings
- Environs and .env file
- Uploaded user avatars
- Django Debug Toolbar configured in development

# Getting it running in development

1. Download the code
2. Use Python 3.8+
3. Configure a virtual environment
4. Install the packages
5. Migrate the DB
6. Configure a superuser
7. Load the data
   - `$ python manage.py loaddata questions`

# Django Trivia – Project structure

- **trivia_site/**
  - trivia_project/
  - apps/
  - static/
  - templates/
  - fixtures/
  - requirements/
  - manage.py

# Django Trivia – Project structure

- trivia_site/
  - **trivia_project/**
    - settings/
      - base.py
      - local.py
      - production.py
    - urls.py
    - wsgi.py
    - asgi.py
  - apps/
  - static/
  - templates/
  - fixtures/
  - requirements/
  - manage.py

# Django Trivia – Project structure

- trivia_site/
  - trivia_project/
  - **apps/**
    - **questions/**
      - migrations/
      - urls.py
      - views.py
      - models.py
      - admin.py
      - apps.py
      - forms.py
    - **users/**
  - **...**

# Django Trivia – Project structure

- trivia_site/
  - trivia_project/
  - apps/
  - **static/**
    - css/
    - images/
    - js/
  - templates/
  - fixtures/
  - manage.py

# Django Trivia – Project structure

- trivia_site/
  - trivia_project/
  - apps/
  - static/
  - **templates/**
    - base.html
    - index.html
    - 404.html
    - questions/
    - users/
    - admin/
    - partials/
      - header.html
      - footer.html

Pearson

# Django Trivia – Project structure

- trivia_site/
  - trivia_project/
  - apps/
  - static/
  - templates/
  - **fixtures/**
    - questions.json  # seed data
  - requirements/
  - manage.py

```
$ python manage.py loaddata questions
```

# Django Trivia – Project structure

- trivia_site/
  - trivia_project/
  - apps/
  - static/
  - templates/
  - fixtures/
  - **requirements/**
    - base.txt
    - local.txt
    - production.txt
  - manage.py

```
$ pip install -r requirements/local.txt
```

# Intermediate Concepts

Deployment settings

# Splitting up settings.py

1. Have a `settings.py` and `local_settings.py`

2. Have a settings folder, containing:
   - `base.py`
   - `production.py`
   - `local.py`

# local_settings.py

- At the bottom of `settings.py`

```
try:
    from .local_settings import *
except ImportError:
    pass
```

- On your machine, create a `local_settings.py` with your local environment specific settings
  - Add `local_settings.py` to `.gitignore`
  - Commit a **`local_settings.py.template`** file to store default config

# base.py

- Create a `base.py` file and a file for each environment

- At the top of each environment file

```
from .base import *
```

- Feel free to set the default environment variables in:
  - `wsgi.py` and `asgi.py`
  - `manage.py`

# With multiple settings files

- Set `DJANGO_SETTINGS_MODULE` environment variable to the correct settings file

# Using environment variables

- You don't want to commit some settings to source control
  - SECRET_KEY
  - Any passwords, keys or tokens
- Some settings are different depending on the environment
  - DB type, location, user, password
  - DEBUG
  - ALLOWED_HOSTS
- Store these either in **local_settings.py** (not committed), **environment variables** or a **.env** file that is not committed to source control

# Using environment variables

- Replace hard-coded settings with values retrieved from environment variables

```
import os

SECRET_KEY = os.environ["DJANGO_SECRET_KEY"]
DEBUG = bool(os.getenv("DJANGO_DEBUG", default=False))
```

- Cons:
  - Default config hard to share with other developers
  - Everything is a string, need to convert types manually

# Using .env files with **environ**

- Replace hard-coded settings with values form a .env file

```
import environ
env = environ.Env()                # Get os environ
env.read_env(BASE_DIR / ".env")  # Read .env file

SECRET_KEY = env("DJANGO_SECRET_KEY")  # Required
DEBUG = env.bool("DJANGO_DEBUG", default=False)
```

- Commit a **.env-example** file for default config
- Variables can either be in environment variables or .env file

# Production settings

- **DEBUG** = False
- **SECRET_KEY**
  - Set it to a unique, unpredictable value, over 50 characters
  - Don't commit to source control, use environment variables
- **ALLOWED_HOSTS**
  - A list of domains that this Django project can serve
- **LOGGING**

# Static and media files

- **Static files** – JS, CSS and images used in your site
- **Media files** – User uploaded content (images, videos, files)

- In ***development***, your static files can be in many locations
  - E.g. in {installed_app}/static/ and any STATICFILES_DIRS

- In ***production***, you run `manage.py collectstatic` to collect them all to one location (STATIC_ROOT)

# Static and media files in production

- Options for serving static and media files:
  1. On the same server as your app
  2. On a different server
  3. Using a cloud service or CDN (Content Delivery Network)

- Serving static files in production

# Static and media files in production

1. **On the same server as your app**
   - Files are served from STATIC_ROOT and/or MEDIA_ROOT locations on the server
   - Ability to do this depends on your web host/server
   - WhiteNoise middleware can help serve static files

2. On a different server

3. Using a cloud service or CDN

# Static and media files in production

1. On the same server as your app

2. **On a different server**
   - Files are transferred from STATIC_ROOT and/or MEDIA_ROOT to the other server after **collectstatic** is run (often using **rsync**)
   - Set the STATIC_URL and/or MEDIA_URL to the other server

3. Using a cloud service or CDN

# Static and media files in production

1. On the same server as your app

2. On a different server

3. **Using a cloud service or CDN**
   - Set the STATIC_URL and/or MEDIA_URL to the other location
   - Set STATICFILES_STORAGE (static) and/or DEFAULT_FILE_STORAGE (media) to use the cloud service
   - For Amazon S3 storage, use Boto3 to automatically transfer files

# Static and media files - notes

- Some hosting solutions, like Heroku, cannot serve media files from the filesystem
  - Use a CDN, like Amazon S3 for media files

- Small-medium web apps often serve static files from the filesystem using Whitenoise

# More resources - Settings

- **Official documentation**
  - Overview
  - Full list of settings
  - Settings organized by topic

- **Medium**: Configuring Django Settings: Best Practices

- **How to's**:
  - Managing static files
  - Deploying static files

# Intermediate Concepts

Django ORM - Models

# Creating models

```
from django.db import models

class Question(models.Model):
    text = models.CharField('question', max_length=255)
    details = models.TextField(blank=True)
```

- Create class that subclasses **models.Model**
- Add model fields
- Override __str__ and __repr__

# Model id's

- When you create a model, an **id** field is automatically created
  - Unless you specify a primary key field
- Default id type is in settings: DEFAULT_AUTO_FIELD
- Access via **object.id** or **object.pk** (pk stands for primary key)
- We often use **pk** (instead of **id**) when retrieving objects, just in case the model uses a custom primary key field
  - e.g. `Question.objects.get(pk=12)`

# Model fields

- Lots of Field types
  - BooleanField, CharField, IntegerField, DateTimeField, TextField
  - DecimalField, DurationField, EmailField, UUIDField, + more
  - ImageField, FileField

- Commonly used field arguments
  - null, default, max_length, unique
  - For DateTimeFields
    - auto_now sets to the current datetime when modified
    - auto_now_add sets to the current datetime when created

# Other fields options

- You can also add validators, help text, error messages, and verbose_name
- These features are used in the Admin and forms
- Full list of model field types and options: https://docs.djangoproject.com/en/4.2/ref/models/fields/

# Blank vs null

- **blank** allows a form field to be blank
- **null** allows null values in the database
- When a **CharField** is left blank in the admin, it gets set to the empty string
- The book **Two Scoops of Django** has a good table that helps determine when to set **blank=True** and **null=True**

*6.4: Django Model Design*

| Field Type | Setting null=True | Setting blank=True |
|---|---|---|
| `CharField`, `TextField`, `SlugField`, `EmailField`, `CommaSeparated-IntegerField`, `UUIDField` | *Okay* if you also have set both `unique=True` and `blank=True`. In this situation, `null=True` is required to avoid unique constraint violations when saving multiple objects with blank values. | *Okay* if you want the corresponding form widget to accept empty values. If you set this, empty values are stored as `NULL` in the database if `null=True` and `unique=True` are also set. Otherwise, they get stored as empty strings. |
| `FileField`, `ImageField` | *Don't do this.* Django stores the path from `MEDIA_ROOT` to the file or to the image in a CharField, so the same pattern applies to FileFields. | *Okay.* The same pattern for CharField applies here. |
| `BooleanField` | *Okay.* | Default is `blank=True`. |
| `IntegerField`, `FloatField`, | *Okay* if you want to be able to set the value to `NULL` in | *Okay* if you want the corresponding form widget to |

# Meta class

- Set *verbose_name* and *verbose_name_plural* of model
- Set *ordering*
- Set db properties over groups of fields, like *index_together* and *unique_together*

- Docs: https://docs.djangoproject.com/en/4.2/ref/models/options/

- You can access `MyModel._meta` to get details about a model
- Even though it is "hidden" it's very commonly used
  - They're working on making it public

# Relationships

```
class Choice(models.Model):
    question = models.ForeignKey(
        Question,
        on_delete=models.CASCADE,
        related_name="choices"
    )
    text = models.CharField(max_length=100)
    is_correct = models.BooleanField(default=False)
    details = models.TextField(blank=True)
```

# Relationship field options

- `on_delete` is required
  - CASCADE, SET_NULL, PROTECT are some common values
- `related_name` determines how to get this model from the related one
  - `question.choices.all()`
  - Default is `question.choice_set.all()`

# Defining relationships

- Many-to-one relationship:
  - Use `ForeignKey`
- Many-to-many relationships:
  - Use `ManyToManyField`
- One-to-one relationships:
  - Use `OneToOneField`

# Resources – ORM Models

- **Official documentation**
  - Overview
  - All DB topics
  - API references
    - Model field types and options
    - Model Meta options

- **LearnDjango:** Django Best Practices: Models

# Intermediate Concepts

Django ORM - Queries

# Work with the Django ORM

- Besides in files, you can use the Django ORM in:
  - The terminal via `$ python manage.py shell`
  - PyCharm's Python console
    - Setup instructions for PyCharm Community
    - Setup instructions for PyCharm Professional
  - Jupyter notebooks using django-extensions

# Using models

- Create an object
- Get an attribute
- Update
- Delete

Examples were shown in Lesson 4

Example usage in **apps.questions.examples.models.py**

- Lines 39 - 65

# Retrieving models

- Get an object by its id
- Get the first or last object
- Get a set of objects
- Filter a set of objects
- Get the number of objects in a set
- Update objects in bulk

Example usage in **apps.questions.examples.models.py**

- Lines 66 - 102

# Querysets

- When Django returns a set of objects, it's called a **Queryset**
- It's like a list, but lazily evaluated
  - The db is only queried when you access an attribute
  - You can't get objects by index e.g. Question.objects.all()**[5]**

- You can make quite advanced queries
  - Look up Q(), F(), aggregation, and conditional expressions
  - Optimize with select_related and prefetch_related
  - Can always write raw SQL

# Managers

- A manager is what is returned if you call Question**.objects**
- You can create custom managers that can store custom queries on your models

- Custom query examples:
  - Question.objects.**with_difficulty("Easy")**
  - Question.objects.**zero_or_many_answers()**

# Resources – ORM Queries

- **Official documentation**
  - Queries overview
  - Aggregation overview
  - List of query methods and aggregation functions

- **Learn Django:** Django ORM and QuerySets Tutorial

# Intermediate Concepts

Django ORM - Migrations

# Migrations

- When a model changes, the database needs to be kept up to date

```
$ ./manage.py makemigrations
$ ./manage.py migrate
```

- You can see the SQL created for a migration

```
$ ./manage.py sqlmigrate <app_name> <migration>
```

e.g. `./manage.py sqlmigrate questions 0003`

# Migrating

- To apply a specific migration

  ```
  $ ./manage.py migrate <app_name> <migration>
  ```

  - This command will also unapply a migration if you select one "in the past"

E.g. `$ ./manage.py migrate questions 0003`

  - If your db is behind 0003, then it will migrate up to 0003
  - If your db is ahead of 0003, it will rollback until 0003

- To target the initial state, use "**zero**" instead of a #
- To "trick" Django into thinking it's at a specific migration without affecting the db, use the **--fake** flag (**migrate --fake <app> <num>**)

# Migrations

- You can run arbitrary Python code in a migration
  - Usually used for updating the data inside your db
  - Tutorial

- For example, if you want to add a non-null field to an existing model with a complex default value
  1. Create the new field, but make it nullable
  2. Run Python code to set the value of the new field for existing objects
  3. Make the field non-null

# Resources – ORM Migrations

- **Official documentation**
  - Overview
  - How to write migrations
  - List of migration operations

- **RealPython:** Django Migrations: A Primer

# Intermediate Concepts

The User Model

# UserModel

- The contrib.auth app comes with a default User model

```
from django.contrib.auth.models import User
```

- Potential problems:
  - Default login is username + password (not email + password)
  - You usually want to add additional fields to the user model

# Custom user models

- Create a custom user inheriting from AbstractUser
- Set `AUTH_USER_MODEL` in settings to point to the custom one
- Use `get_user_model()` to get the correct model

```
from django.contrib.auth import get_user_model
UserModel = get_user_model()
```

Example usage in **apps.users.models.py**

- Documentation - custom auth user

# Inheritance

- Inheriting Model classes in Django can be tricky
- Safest way is to create an abstract base class (not saved in db) and concrete child classes (saved in separate tables)
- You can't query over all subclasses easily, but it saves a lot of trouble in the long run

- Set `abstract=True` in the Meta class

# User login/logout/registration

- Login/logout tutorial
  - https://learndjango.com/tutorials/django-login-and-logout-tutorial
- Registration tutorial
  - https://learndjango.com/tutorials/django-signup-tutorial
- Use All Auth to handle 3rd party authentication
  - https://django-allauth.readthedocs.io/en/latest/index.html

# Intermediate Concepts

URLs

# Get a URL by name

```
app_name = "users"
urlpatterns = [
    path("me/", view=views.profile, name="profile"),
]
```

- **In Python files**

```
from django.urls import reverse
url = reverse("users:profile")  # "/users/me/"
```

- **In HTML files**

```
{% url "users:profile" %}
```

e.g. `<a href="{% url 'users:profile' %}">My profile</a>`

# Get a URL by name

```
app_name = "users"
urlpatterns = [
    path("me/", view=views.profile, name="profile"),
]
```

Here, "users" comes from the main urls.py where "users.urls" is included

- **In Python files**

```
from django.urls import reverse
url = reverse("users:profile")  # "/users/me/"
```

- **In HTML files**

```
{% url "users:profile" %}
```

e.g. `<a href="{% url 'users:profile' %}">My profile</a>`

# Accepting arguments

- Use <> to capture URL arguments and pass them to the view

```
path('users/<username>/', views.profile)
```

- Arguments are strings by default, but you can specify the type

```
path('users/<int:pk>/, views.profile)
```

# Regular expressions

- You can also capture arguments via regular expressions using `re_path()`

```
re_path(r'^users/?P<username>\w+/$', views.profile)
```

- Note:
  - This is the same as **url()** in older versions of Django
  - The new **path()** function is easier to read and sufficient for most cases
- Intro to regular expressions (video)
- Regular expressions cheat sheet

# Get a URL by name with arguments

```
path("<username>/", view=views.detail, name="profile")
```

- **In Python files**

```
reverse("users:profile", args=["user1"])
reverse("users:profile", kwargs={"username": "user1"})
```

- **In HTML files**

```
{% url 'users:profile' 'admin' %}          # Constant
{% url 'users:profile' user.username %}    # Variable
{% url 'users:profile' username='admin' %} # Kwargs
```

# Combining URLconfs with include()

- URL lists can be combined

```
urlpatterns = [
    path("users/", include("apps.users.urls")),
]
```

- When Django encounters **include()**, it:
  - chops off whatever part of the URL matched up to that point
  - sends the remaining string to the included URLconf to match

# Media files

- If you use any sort of **FileField** in your models, those are considered media files that a user can upload

- In **settings.py** (these might be different in production)

```
MEDIA_URL = '/media/'
MEDIA_ROOT = BASE_DIR / 'media'
```

- In your main **urls.py**

```
from django.conf import settings
from django.conf.urls.static import static

urlpatterns = [  # ... other urls
] + [static(settings.MEDIA_URL, document_root=settings.MEDIA_ROOT)]
```

# Resources

- **Official documentation**
  - Overview
  - URL functions
  - URL resolver functions

# Intermediate Concepts

Views

# A simple view function

```
def index(request):
    some_html = """
    <h1>Welcome to my site!</h1>
    <p>Have a look around</p>
    """
    return HttpResponse(content=some_html, status=200)
```

# View function

- When a URL path gets matched:
  - Django calls the corresponding view function
  - Passes it the request and any captured arguments

- The view function might:
  - return some HTML, JSON data or an exception page
  - handle a GET or POST request differently
  - update and/or retrieve data from the database (via models)
  - require certain permissions from the user

# The request object

- An **HttpRequest** has:
  - A user (as long as you're using the Django auth middleware)
    - Will be an anonymous user if they're not logged in
  - body – content of the request
  - GET/POST – query parameters
  - Headers – metadata
  - request URL

# View responses

- An **HttpResponse** has:
  - A status code (like 200 Okay)
  - Some content (like HTML or JSON)
  - Some metadata (in headers)

# View complexity

- Because certain functionality is common to many views, there are a lot of **shortcuts** you can use to create view functions
  - Django shortcuts, like **render()** and **get_object_or_404()**
  - Class based views, like **CreateView**
  - View mixins, like **LoginRequiredMixin**
  - View decorators, like **login_required**

- The more shortcuts you use, the more base knowledge is required to read your code and write your views

- The most basic view function just takes in an HTTP request and returns an HTTP response

# Shortcut comparison

```python
# trivia_site/apps/questions/views.py

def question_view_1(request, pk):
    q = get_object_or_404(Question, pk=pk)

    context = {"question": q}

    return render(
        request,
        "questions/question.html",
        context=context)
```

```python
def question_view_2(request, pk):
    try:
        q = Question.objects.get(pk=pk)
    except Question.DoesNotExist:
        raise Http404('Not found')

    context = {"question": q}

    content = loader.render_to_string(
        "questions/question.html",
        context,
        request)

    return HttpResponse(
        content,
        content_type="text/html",
        status=200)
```

# Shortcut comparison

```python
# trivia_site/apps/questions/views.py

def question_view_1(request, pk):
    q = get_object_or_404(Question, pk=pk)

    context = {"question": q}

    return render(
        request,
        "questions/question.html",
        context=context)
```

```python
def question_view_2(request, pk):
    try:
        q = Question.objects.get(pk=pk)
    except Question.DoesNotExist:
        raise Http404('Not found')

    context = {"question": q}

    content = loader.render_to_string(
        "questions/question.html",
        context,
        request)

    return HttpResponse(
        content,
        content_type="text/html",
        status=200)
```

# Shortcut comparison

```
# trivia_site/apps/questions/views.py

def question_view_1(request, pk):
    q = get_object_or_404(Question, pk=pk)

    context = {"question": q}

    return render(
        request,
        "questions/question.html",
        context=context)
```

```
def question_view_2(request, pk):
    try:
        q = Question.objects.get(pk=pk)
    except Question.DoesNotExist:
        raise Http404('Not found')

    context = {"question": q}

    content = loader.render_to_string(
        "questions/question.html",
        context,
        request)

    return HttpResponse(
        content,
        content_type="text/html",
        status=200)
```

**TemplateResponse()** is similar to **render()**, but remembers the template it came from

# A simple view function

```
def index(request):
    some_html = """
    <h1>Welcome to my site!</h1>
    <p>Have a look around</p>
    """
    return HttpResponse(content=some_html, status=200)
```

# A more realistic view function

```
@require_GET
@login_required
def list_view(request):
    if request.user.is_staff:
        users = User.objects.all()
    else:
        users = User.objects.filter(is_staff=False)
    context = {"users": users}
    return render(request, "users/user_list.html", context=context)
```

# View decorators

- Before a view function is called, you can do some default request handling

- Examples:
  - Require a user be logged in, redirect them to a login view if not
    - Use @login_required
  - Only accept certain HTTP methods, or return a 405 error
    - Use @require_GET, @require_POST or @require_http_methods

# A more realistic view function

```
@require_GET
@login_required
def list_view(request):
    if request.user.is_staff:
        users = User.objects.all()
    else:
        users = User.objects.filter(is_staff=False)
    context = {"users": users}
    return render(request, "users/user_list.html", context=context)
```

Only accept GET requests, return a 405 if other method used

# A more realistic view function

```
@require_GET
@login_required
def list_view(request):
    if request.user.is_staff:
        users = User.objects.all()
    else:
        users = User.objects.filter(is_staff=False)
    context = {"users": users}
    return render(request, "users/user_list.html", context=context)
```

**If a user isn't logged in, send them to the login page first, then redirect to this view**

# A more realistic view function

```
@require_GET
@login_required
def list_view(request):
    if request.user.is_staff:
        users = User.objects.all()
    else:
        users = User.objects.filter(is_staff=False)
    context = {"users": users}
    return render(request, "users/user_list.html", context=context)
```

If the logged in user is staff, show the all users

# A more realistic view function

```
@require_GET
@login_required
def list_view(request):
    if request.user.is_staff:
        users = User.objects.all()
    else:
        users = User.objects.filter(is_staff=False)
    context = {"users": users}
    return render(request, "users/user_list.html", context=context)
```

Otherwise, show a filtered list

# A more realistic view function

```
@require_GET
@login_required
def list_view(request):
    if request.user.is_staff:
        users = User.objects.all()
    else:
        users = User.objects.filter(is_staff=False)
    context = {"users": users}
    return render(request, "users/user_list.html", context=context)
```

**Render a template with a context and return the HttpResponse**

# More view logic features

- **Messages**
  - One-time notification messages
  - E.g. your form submission was successful
- **Sessions**
  - Store and retrieve arbitrary data on a per-site-visitor basis
  - Data stored on the server side
  - Cookies contain a session ID – not the data itself
- **Handle Django Forms**
  - Input validation and cleaning easier than DIY
- apps/questions/views.py shows these features

# Exceptions

- When returning an **HttpResponse**, you can pass in a custom status_code or return a subclass, like **HttpResponseNotFound**
  - But these require you to pass in the content as well

- In order to return consistent error pages, you can raise an **Http404** exception so Django uses your 404.html template

- **Note**: to test error templates, you must make DEBUG=False or create special URLs to serve them
  - If `DEBUG = False`, static files (like styles.css) will fail to load
  - Use `./manage.py runserver --insecure` to serve static files locally

# Resources

- **Official documentation**
  - Overview
  - Request and Response objects
  - Authentication decorators

- **Django Views the Right Way:** The Pattern
  - An opinionated guide
- **Simple is better than complex**: Working with Django View Decorators
  - Creating custom decorators

# Resources

- **Messages**
  - Documentation
- **Sessions**
  - Documentation
- **Emails**
  - Documentation

# Intermediate Concepts

Class Based Views

# Function vs class-based views

- I've included some examples in the **users** app of class-based views (CBVs)
- Some Django users love CBVs and some prefer FBVs (function-based views)
- In general, I think FBVs are easier to read, understand, and write and recommend them for beginners
- As a good starting place, you can inherit from the base View

# The basic View

- The previous function-based view can be rewritten as:

```
class ListView(LoginRequiredMixin, View):
    def get(self, request):
        if request.user.is_staff:
            users = User.objects.all()
        else:
            users = User.objects.filter(is_staff=False)
        context = {"users": users}
        return render(request, "users/user_list.html", context=context)
```

- Link it to the url path by using **ListView.as_view()** to get the function

# The basic View

- The previous function-based view can be rewritten as:

Inherit from the basic View class

```
class ListView(LoginRequiredMixin, View):
    def get(self, request):
        if request.user.is_staff:
            users = User.objects.all()
        else:
            users = User.objects.filter(is_staff=False)
        context = {"users": users}
        return render(request, "users/user_list.html", context=context)
```

# The basic View

- The previous function-based view can be rewritten as:

Use a mixin instead of a decorator

```
class ListView(LoginRequiredMixin, View):
    def get(self, request):
        if request.user.is_staff:
            users = User.objects.all()
        else:
            users = User.objects.filter(is_staff=False)
        context = {"users": users}
        return render(request, "users/user_list.html", context=context)
```

# The basic View

- The previous function-based view can be rewritten as:

```
class ListView(LoginRequiredMixin, View):
    def get(self, request):
        if request.user.is_staff:
            users = User.objects.all()
        else:
            users = User.objects.filter(is_staff=False)
        context = {"users": users}
        return render(request, "users/user_list.html", context=context)
```

Define a get method to accept GETs

Don't define a post to return a 405 on POSTs

# CBV Gotchas

- The order of mixins matters!
  - class ListView(**LoginRequiredMixin, View**) works ✅
  - class ListView(**View, LoginRequiredMixin**) doesn't work ❌
  - Resolution order reads left to right, so the CBV should be last since mixins should resolve before the base class
  - Detailed explanation

# Resources – Class-Based Views

- **Official documentation**
  - Overview
  - List of View classes

- **Learn Class-Based Views**: A comprehensive Django CBV guide

# Intermediate Concepts

Django Admin

# Overall design

- Index
  - Apps
    - Models
      - List view
      - Add view
      - Change view

# Defining ModelAdmin

```
@admin.register(Visit)
class VisitsAdmin(admin.ModelAdmin):
    list_display = ['__str__', 'page', 'username', 'timestamp']
    list_filter = ['page', 'username', 'timestamp']
    search_fields = ['page', 'username']
    readonly_fields = ['timestamp']
```

# List view

Select user to change | Django

localhost:8000/admin/users/user/

Incognito Update

Django administration

WELCOME, ARIANNE. VIEW SITE / CHANGE PASSWORD / LOG OUT

Home › Users › Users

Select user to change

ADD USER +

Search

Action: --------- Go 0 of 2 selected

| | USERNAME | EMAIL ADDRESS | NAME | SUPERUSER STATUS | STAFF STATUS | ACTIVE |
|---|---|---|---|---|---|---|
| ☐ | admin | admin@test.com | Arianne | ✓ | ✓ | ✓ |
| ☐ | test1 | test1@test.com | Test User 1 | ✗ | ✗ | ✓ |

2 users

FILTER

By superuser status
All
Yes
No

By staff status
All
Yes
No

By active
All
Yes
No

# List view

- View list of models

- Common options:
  - list_display: which columns to show
  - list_filter: which fields to filter by
  - search_fields: which fields to search on
  - ordering: default order field of list
  - actions: bulk actions available
  - list_display_links: which fields link to change view

# List view

Select user to change | Django

localhost:8000/admin/users/user/

Incognito Update

Django administration

WELCOME, ARIANNE. VIEW SITE / CHANGE PASSWORD / LOG OUT

Home › Users › Users

Select user to change

ADD USER +

Search

Action: --------- Go 0 of 2 selected

| | USERNAME | EMAIL ADDRESS | NAME | SUPERUSER STATUS | STAFF STATUS | ACTIVE |
|---|---|---|---|---|---|---|
| ☐ | admin | admin@test.com | Arianne | ✓ | ✓ | ✓ |
| ☐ | test1 | test1@test.com | Test User 1 | ✗ | ✗ | ✓ |

2 users

FILTER

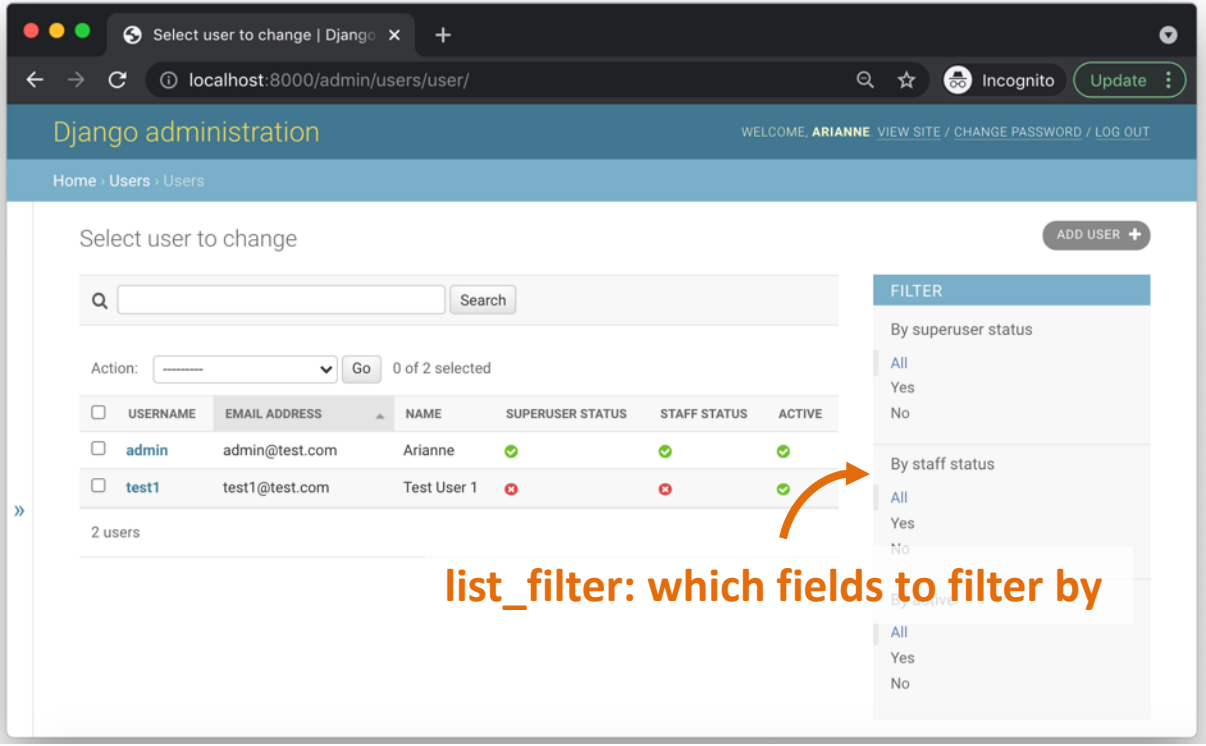
By superuser status
All
Yes
No

By staff status
All
Yes
No

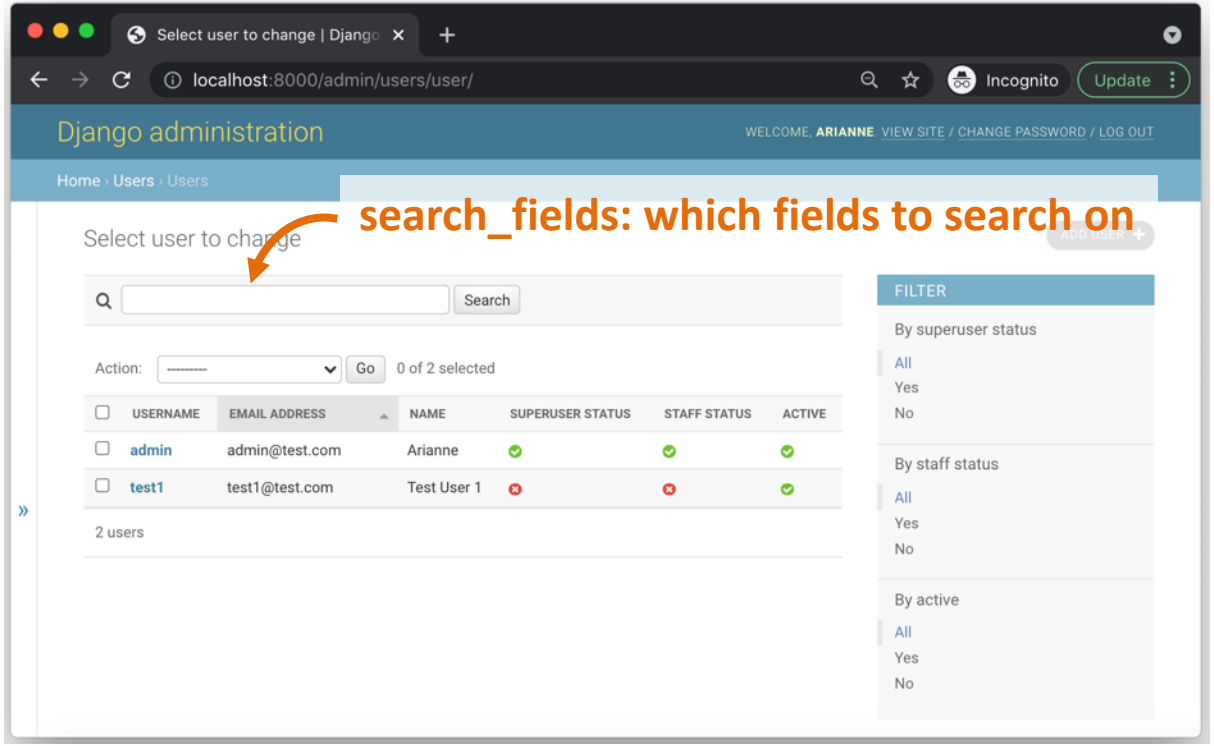
By active
All
Yes
No

**list_display: which fields to show**

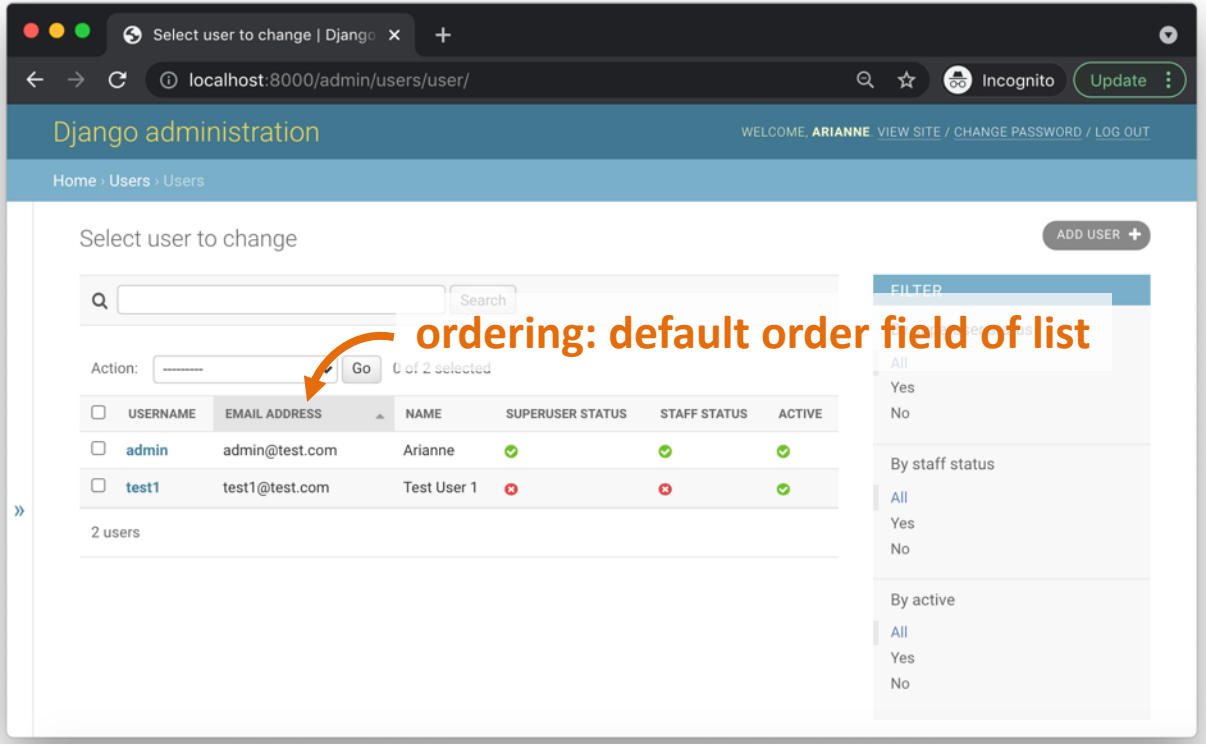
# List view


Select user to change | Django
localhost:8000/admin/users/user/
Incognito
Update
Django administration
WELCOME, ARIANNE. VIEW SITE / CHANGE PASSWORD / LOG OUT
Home › Users › Users
Select user to change
ADD USER
Search
Action: --------
Go
0 of 2 selected
USERNAME
EMAIL ADDRESS
NAME
SUPERUSER STATUS
STAFF STATUS
ACTIVE
admin
admin@test.com
Arianne
test1
test1@test.com
Test User 1
2 users
FILTER
By superuser status
All
Yes
No
By staff status
All
Yes
No
All
Yes
No
list_filter: which fields to filter by

# List view


Select user to change | Django
localhost:8000/admin/users/user/
Incognito
Update
Django administration
WELCOME, ARIANNE. VIEW SITE / CHANGE PASSWORD / LOG OUT
Home › Users › Users
search_fields: which fields to search on
Select user to change
Search
Action:
Go
0 of 2 selected
USERNAME
EMAIL ADDRESS
NAME
SUPERUSER STATUS
STAFF STATUS
ACTIVE
admin
admin@test.com
Arianne
test1
test1@test.com
Test User 1
2 users
FILTER
By superuser status
All
Yes
No
By staff status
All
Yes
No
By active
All
Yes
No

# List view


Select user to change | Django
localhost:8000/admin/users/user/
Incognito
Update
Django administration
WELCOME, ARIANNE. VIEW SITE / CHANGE PASSWORD / LOG OUT
Home › Users › Users
Select user to change
ADD USER
Search
ordering: default order field of list
Action:
Go
0 of 2 selected
USERNAME
EMAIL ADDRESS
NAME
SUPERUSER STATUS
STAFF STATUS
ACTIVE
admin
admin@test.com
Arianne
test1
test1@test.com
Test User 1
2 users
FILTER
All
Yes
No
By staff status
All
Yes
No
By active
All
Yes
No

# List view

Select user to change | Django

localhost:8000/admin/users/user/

Incognito Update

Django administration

WELCOME, ARIANNE. VIEW SITE / CHANGE PASSWORD / LOG OUT

Home › Users › Users

Select user to change

ADD USER +

actions: bulk actions available

Search

Action: --------- Go 0 of 2 selected

| | USERNAME | EMAIL ADDRESS | NAME | SUPERUSER STATUS | STAFF STATUS | ACTIVE |
|---|---|---|---|---|---|---|
| ☐ | admin | admin@test.com | Arianne | ✓ | ✓ | ✓ |
| ☐ | test1 | test1@test.com | Test User 1 | ✗ | ✗ | ✓ |

2 users

FILTER

By superuser status
All
Yes
No

By staff status
All
Yes
No

By active
All
Yes
No

# List view

Select user to change | Django

localhost:8000/admin/users/user/

Incognito Update

Django administration

WELCOME, ARIANNE. VIEW SITE / CHANGE PASSWORD / LOG OUT

Home › Users › Users

Select user to change

ADD USER +

Search

Action: --------- Go 0 of 2 selected

| | USERNAME | EMAIL ADDRESS | NAME | SUPERUSER STATUS | STAFF STATUS | ACTIVE |
|---|---|---|---|---|---|---|
| ☐ | admin | admin@test.com | Arianne | ✓ | ✓ | ✓ |
| ☐ | test1 | test1@test.com | Test User 1 | ✗ | ✗ | ✓ |

2 users

FILTER

By superuser status
All
Yes
No

By staff status
All
Yes
No

By active
All
Yes
No

**list_display_links: which fields link to change view**

# Change/add view

- Create or edit your models here

- Common options:
  - readonly_fields
  - fieldsets: define groupings of fields
  - inlines: edit models on the same page as a parent model

# Readonly fields

Home › Visits › Visits › Visit 4

Change visit

HISTORY

**Visit 4**

| | |
|---|---|
| **Page:** | favicon.ico |
| **Username:** | admin |
| Timestamp: | Aug. 3, 2023, 6:24 a.m. |

SAVE | Save and add another | Save and continue editing | Delete

dj_test_2/visits/admin.py

# Fieldsets

Change user

HISTORY VIEW ON SITE

**test@test.com**

User

**Name:** admin

Avatar: **Currently:** avatars/IMG_1375_GHYpyl9.jpeg Clear

**Change:** Choose File No file chosen

**Username:** admin

Required. 150 characters or fewer. Letters, digits and @/./+/-/_ only.

**Password:** pbkdf2_sha256$600000$4t0FO7a2f8Ujuf.

Personal info

First name:

Last name:

**Email address:** test@test.com

Permissions

Active

Designates whether this user should be treated as active. Unselect this instead of deleting accounts.

trivia_site/apps/users/admin.py

# Inlines

Change Question

HISTORY VIEW ON SITE

**Question: What does ORM stand for?**

**Question:** What does ORM stand for?

Details:

Difficulty: ---------

CHOICES

| CHOICE | IS CORRECT | DETAILS |
|---|---|---|
| Choice: Object-Role Modeling | | |
| Object-Role Modeling | ☐ | |
| Choice: Object-Relationship Model | | |
| Object-Relationship Model | ☐ | |

trivia_site/apps/questions/admin.py

# Permissions

- Django comes with permissions and group permissions
- You can configure what permissions a user/group has, per model:
  - Add
  - Edit
  - Delete
  - Readonly

# Customizing style and templates

- Add folders called **admin** in your **templates** and **static/css/ folders**
- When Django tries to load the templates and styles for the admin site, it will check your local files first before looking in the admin app's folders
- You can override any of the admin's default files this way

# Resources

- **Official documentation**
  - Overview and API reference

- **RealPython**: Customizing the admin
- **RealPython**: Manage users in Django Admin

# Intermediate Concepts

Front-end

# Front-end for Django

- The easiest set-up is to use the **Django Template Language** to create your HTML pages
- You can swap it for the **Jinja2**, and can run arbitrary Python code, or write your own template backend
- This method doesn't allow you to create single-page apps
  - Limited to the old **page -> click -> request -> new page** flow

# Front-end for Django

- If you want to create an interactive, single-page app, you can still use Django as the backend

- Create a REST or GraphQL API to interact with a JS front-end framework, like Angular, React or Vue
  - REST API – Use Django Rest Framework (DRF)
  - GraphQL API – Use Graphene-Django

- Middle ground:
  - JQuery and custom JavaScript
  - HTMX

# CSS frameworks and Django

- Use a CSS framework to make your site responsive and interactive
- Bootstrap is common, but other options are Foundation, Tailwind, Bulma, and many more
- One thing to consider is that Django forms can be created like so:
  - `{{ form.as_p }}`
  - If you want your form to conform to the CSS framework you're using, try to find a django extension for it

# Bulma – CSS Framework

- In the sample Trivia project, I'm using **Bulma**
  - https://bulma.io/

- To handle my Django forms, I'm using **django-bulma**
  - https://github.com/timonweb/django-bulma

You can use one of the **6 main colors**:

- `is-primary`
- `is-link`
- `is-info`
- `is-success`
- `is-warning`
- `is-danger`

Button
Button
Button
Button
Button
Button

```
<a class="button is-primary">
  Button
</a>
<a class="button is-link">
  Button
</a>
<a class="button is-info">
  Button
</a>
<a class="button is-success">
  Button
</a>
<a class="button is-warning">
  Button
</a>
<a class="button is-danger">
  Button
</a>
```

# Resources – Jinja Templating

- **Official documentation**:
  - Configuring the template engine
  - Differences between Django and Jinja

- **RealPython**: Primer of Jinja Templating

# Resources

- **Vue.js + Django**:
  - Udemy course
  - Django Waves tutorial

- **React.js + Django**:
  - Udemy course
  - Digital Ocean tutorial

# Intermediate Concepts

Django Template Language

# Templates

- Templates are just HTML files with special syntax that the Django Template Language (DTL) uses to replace with dynamic content

- Variable: `{{ var }}`
- Filter: `{{ var|filter }}`
- Tag: `{% tag %}`
- Comment: `{# comment #}`

# Example template

```
{% extends "base.html" %}

{% block content %}

  <h1>{{ user.name|upper }}</h1>

  {% for field in user_fields %}
    {% include "partials/field_display.html" %}
  {% endfor %}

{% endblock %}
```

# Variables

- Some data you can use in your template
  - **request**, **user**, and anything in your **context** dict

- You can access attributes of the object with dot-notation
  - **Model field:** `{{ user.email }}`
  - **Model property:** `{{ question.random_choices }}`
  - **Model method (no arguments):** `{{ user.get_absolute_url }}`
  - **Dictionary value:** `{{ dict.key }}`
  - **List index:** `{{ list.0 }}` (no negative index)

# Variables - cont

- To see available variables, add this to your template:

```
<pre> {% filter force_escape %} {% debug %} {% endfilter %} </pre>
```

  - Only look at the top section for useful info (not long list of modules)

- If you need to call a model method with arguments, you'd need to create a template tag or use a different templating library, like Jinja2
  - Read more

# Filters

- Allow you to modify variables for display purposes
  - `{{ user.name|lower }}`
  - `{{ my_list|last }}`
  - `{{ users|length }}`

- You can pass arguments in
  - `{{ a_value|default:"nothing" }}`

- All built-in filters
- Creating custom filters

# Tags

- Provide logic for templates
  - `{% for u in users %} ... {% endfor %}`
  - `{% if user.is_staff %} ... {% endif %}`
  - `{% extends "base.html" %}`
  - `{% block title %} ... {% endblock %}`
  - `{% include "partials/header.html" %}`

- All built-in tags
- Creating custom tags

# Example template

Inherit from the file base.html

```
{% extends "base.html" %}

{% block content %}

  <h1>{{ user.name|upper }}</h1>

  {% for field in user_fields %}
    {% include "partials/field_display.html" %}
  {% endfor %}

{% endblock %}
```

# Template inheritance

- Usually, you define a base template that includes your sites:
  - CSS styles
  - JavaScript libraries
  - general layout (header, footer, content wrappers)

- Create a `base.html` file
- Every other file will extend the base file
- Customize parts of the base file using `{% block %}` tags

# Simple base.html

```
<head>
  <title>Django Trivia</title>
  <link href="/static/css/style.css" type="text/css" rel="stylesheet">
</head>
<body>
  <section class="main">
  {% block content %}
    Put some custom content here
  {% endblock %}
  </section>
  <script src="/static/js/project.js"></script>
</body>
</html>
```

# Simple base.html

```
<head>
  <title>Django Trivia</title>
  <link href="/static/css/style.css" type="text/css" rel="stylesheet">
</head>
<body>
  <section class="main">
  {% block content %}
    Put some custom content here
  {% endblock %}
  </section>
  <script src="/static/js/project.js"></script>
</body>
</html>
```

Add your CSS styles

# Simple base.html

```
<head>
  <title>Django Trivia</title>
  <link href="/static/css/style.css" type="text/css" rel="stylesheet">
</head>
<body>
  <section class="main">
  {% block content %}
    Put some custom content here
  {% endblock %}
  </section>
  <script src="/static/js/project.js"></script>
</body>
</html>
```

Add your JavaScript

# Simple base.html

```
<head>
  <title>Django Trivia</title>
  <link href="/static/css/style.css" type="text/css" rel="stylesheet">
</head>
<body>
  <section class="main">
  {% block content %}
    Put some custom content here
  {% endblock %}
  </section>
  <script src="/static/js/project.js"></script>
</body>
</html>
```

Define a block that can be customized

# Simple base.html

```
<head>
  <title>Django Trivia</title>
  <link href="/static/css/style.css" type="text/css" rel="stylesheet">
</head>
<body>
  <section class="main">
  {% block content %}
    Put some custom content here
  {% endblock %}
  </section>
  <script src="/static/js/project.js"></script>
</body>
</html>
```

This will be replaced if a file that extends base redefines the “content” block

# Template inheritance

- Then in your child page (like `index.html`)
  - Override any **block** tags you want

```
{% extends "base.html" %}

{% block content %}
  <h1>Hello world!</h1>
  <p>Welcome to my website</p>
{% endblock %}
```

**Everything in here will replace whatever is in the base.html block with name "content"**

- Docs – Template inheritance

# Example template

```
{% extends "base.html" %}

{% block content %}

  <h1>{{ user.name|upper }}</h1>

  {% for field in user_fields %}
    {% include "partials/field_display.html" %}
  {% endfor %}

{% endblock %}
```

Override base.html's content

# Example template

```
{% extends "base.html" %}

{% block content %}

  <h1>{{ user.name|upper }}</h1>

  {% for field in user_fields %}
    {% include "partials/field_display.html" %}
  {% endfor %}

{% endblock %}
```

Print the variable "user.name" and transform it using the "upper" filter

# Example template

```
{% extends "base.html" %}

{% block content %}

  <h1>{{ user.name|upper }}</h1>

  {% for field in user_fields %}
    {% include "partials/field_display.html" %}
  {% endfor %}

{% endblock %}
```

Loop through a list of items, e.g. user data

# Example template

```
{% extends "base.html" %}

{% block content %}

  <h1>{{ user.name|upper }}</h1>

  {% for field in user_fields %}
    {% include "partials/field_display.html" %}
  {% endfor %}

{% endblock %}
```

Insert another template here

# Include

- These mini-templates are often called "**partials**" and are put in a special folder

- The included template will use the same context as the original template, as well as variables created in a for loop

- You can pass additional variables to partials:

```
{% include "header.html" with admin=user.is_staff %}
```

# Resources

- **Official documentation**
  - Overview
  - How to create custom tags
  - Template documentation
    - Django Template Language
    - All built-in tags
  - Translation overview

- **Codium.ai**: Django Templates: Best Practices for Web Development

# Intermediate Concepts

Forms

# Django Forms

- You can define forms in Django using Python
- Render them in the template in different formats
  - `{{ form.as_div }}`
  - `{{ form.as_p }}`
  - `{{ form.as_ul }}`
  - `{{ form.as_table }}`

- It can take a lot of custom CSS to make any of these look good

- Use a CSS framework if you want easy form styling
  - Popular option: django-crispy-forms
    - `{{ form|crispy }}`
  - I'm using django-bulma with Bulma
    - `{{ form|bulma }}`

# Form

- You can create forms by defining fields and widgets

```
from django import forms

class ContactForm(forms.Form):
    subject = forms.CharField(max_length=100)
    message = forms.CharField(widget=forms.Textarea)
    sender = forms.EmailField()
```

# ModelForm

- You can also create them magically from models

```
class QuestionForm(ModelForm):
    class Meta:
        model = Question
        fields = ("text", "details")
```

# HTML

- In a template, you can use `{{ form.as_div }}` to render a form

- Or you can render them manually by access fields individually

```
{% for field in form %}
    <div class="my-field-class">
        {{ field.errors }}
        {{ field.label_tag }} {{ field }}
    </div>
{% endfor %}
```

- It's hard to add classes to your field labels and inputs, so:
  - Use an extension built for your CSS framework (like **django-crispy-forms**)
  - Use a generic extension (like **django-widget-tweaks**) to add your own classes

# Widgets

- You can also customize how a form field is displayed in HTML by editing its corresponding widget on the Python side

```
class QuestionForm(forms.ModelForm):
    class Meta:
        model = Question
        fields = ("text", "details")
        widgets = {
            "details": forms.Textarea(attrs={"rows": 3})
        }
```

# Cleaning

- Your form objects have certain methods that you can use and overwrite, like:
  - `form.clean()`
  - `form.clean_<field>()`
  - `form.save()`

- See trivia_site/apps/users/forms.py

# Working with forms in View

- Use Class Based Views, like CreateView or UpdateView
  - See UserSignUp and UserUpdateView in trivia_site/apps/users/views.py

- Use view functions
  - If method == ‘GET’, return a template that renders the form
  - If method == ‘POST’:
    - Instantiate a form with the POST data
    - Check if it’s valid
    - Save the data or return validation errors
  - See question_update() in trivia_site/apps/questions/views.py

# Resources

- **Official documentation**
  - Overview
  - Form documentation
    - Basic forms
    - Model forms
    - Form field types and options

- **Simple is better than complex**: How to Render Django Form Manually

# Summary

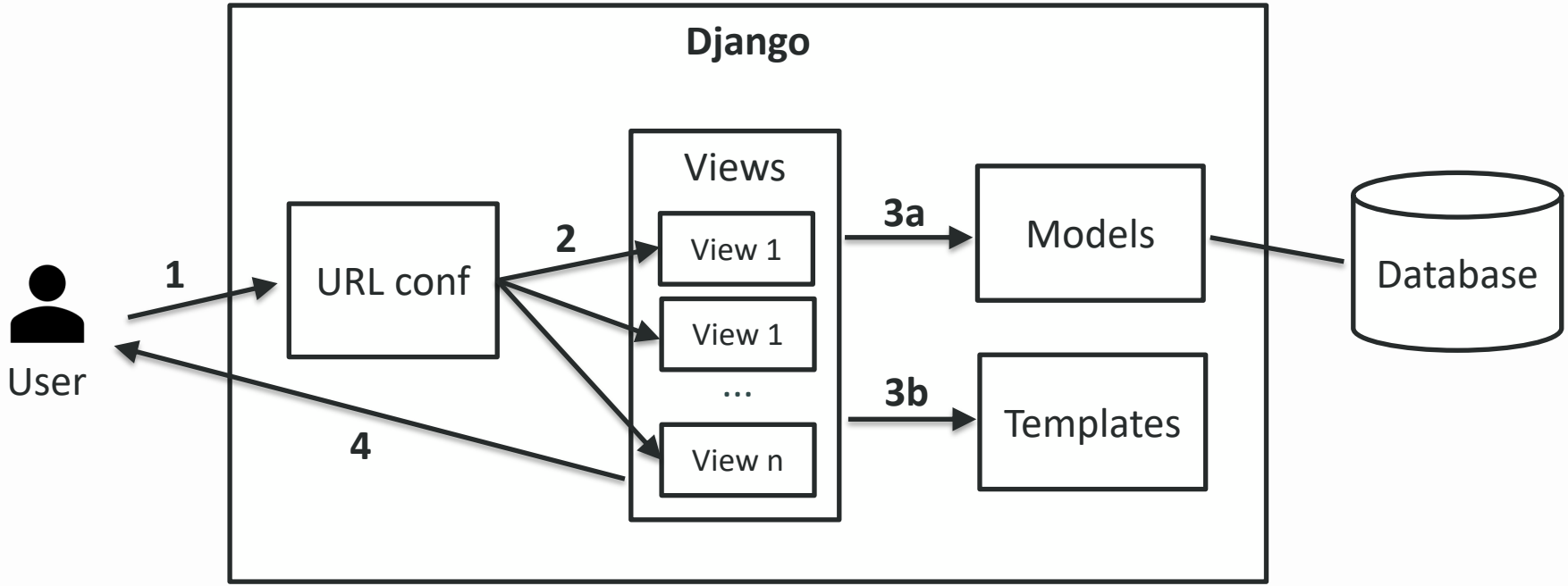
# You learned

- Django is a full-featured web framework written in Python
  - Security, admin, sessions, and user management built-in
- Projects usually have many apps
  - Modular, can build many features quickly and concurrently

# You learned

- Apps use a MVT (Model-View-Template) architecture
- Models and DB can be managed with the Django ORM
- Back-end:
  - URLs + Views handles the endpoints
  - Can use frameworks like Django REST Framework or Graphene for APIs
- Front-end:
  - Templates via Django Template Language or Jinja2
  - Use separate front-end framework like React or Vue.js

# Django architecture overview



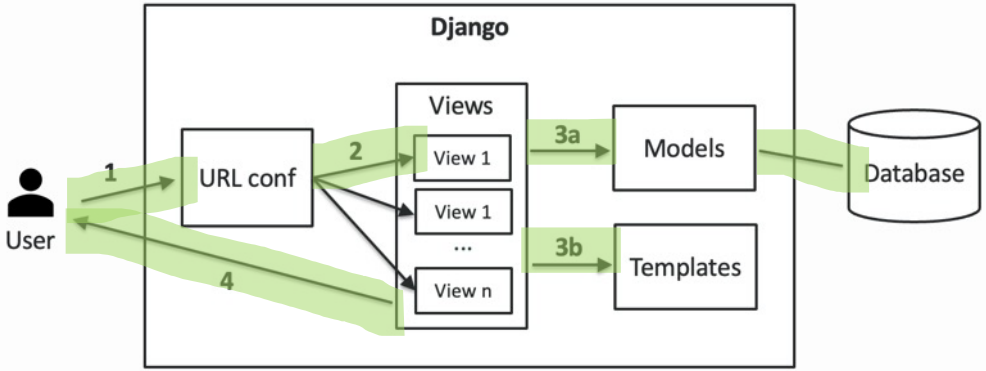
# Django architecture

1. User sends an HTTP **request** to Django

2. **URL conf**iguration, contained in urls.py, selects a View to handle the request

3. The **View** gets the request and
   a) Talks to a database via the **Models**
   b) Renders an HTML **Template**

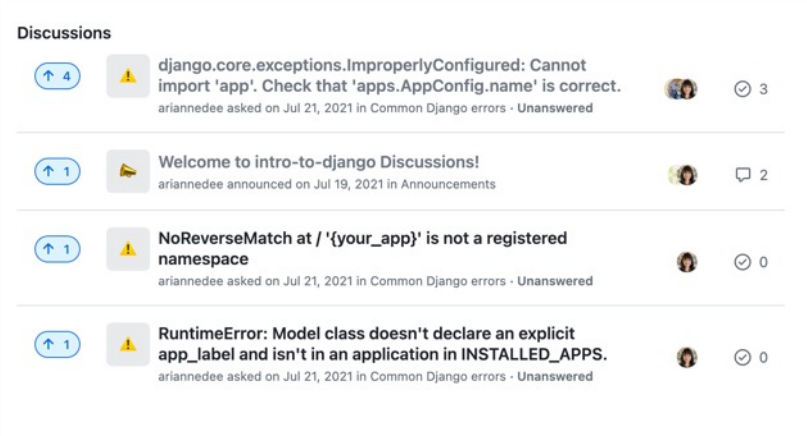
4. The View returns an **HttpResponse** which gets sent to the client to be rendered as a web page in the browser



# Deployment

- Review the Advanced Settings lesson
  - Set DEBUG = False
  - Decide how to serve static and media files
  - Host files in cloud storage or separate server (optional)
  - Configure caching, logging, production database, etc
  - Keep secrets secret
  - Use different settings for different environments

# Django Discussion




- Feel free to post any Django related questions you have to the GitHub Discussion page
- https://github.com/ariannedee/intro-to-django/discussions
- I would like to have set of common errors there to help people learning Django

# Learn more

- Learn Class-based views

- REST APIs in Django Rest Framework
  - Intro tutorial
  - Video course
- GraphQL APIs in Graphene
  - Docs
  - Video tutorial

# More Django Resources

- Awesome Django: https://awesomedjango.org/
  - Useful Django libraries and resources

- Tutorials:
  - **RealPython**: Build a Social Network With Django
  - **Django Girls**: Official Tutorial
  - **Mastering Django**:
    - Django Tutorial For Beginners
    - Mastering Django Tutorial

# Some good Django reading

- Django design philosophy
- General discussion of Django from developers
- Django Class-Based Views were a mistake

# Recommended follow-up

**Videos**

- Web Development in Python with Django: Building Backend Web Applications and APIs with Django
  - Andrew Pinkham

**Books**

- A Wedge of Django – Daniel and Audrey Roy Greenfield
- Mastering Django – Nigel George

# Thanks!

Questions?

Email me at

arianne.dee.studios@gmail.com